

Automatic Design of Interpretable Control Laws Through Parametrized Genetic Programming with Adjoint State Method Gradient Evaluation

Francesco Marchetti^a, Gloria Pietropolli^b, Federico Julian Camerota
Verdù^b, Mauro Castelli^c, Edmondo Minisci^a

^a*Intelligent Computational Engineering Laboratory (ICE-Lab), Department of
Mechanical and Aerospace Engineering, University of Strathclyde, 75 Montrose
Street, Glasgow, G11XJ, Scotland, United Kingdom*

^b*Dipartimento di Matematica e Geoscienze, Università degli Studi di Trieste, H2bis
Building, Via Alfonso Valerio 12/1, Trieste, 34127, TS, Italy*

^c*NOVA Information Management School (NOVA IMS), Universidade NOVA de
Lisboa, Campus de Campolide, Lisbon, 1070-312, Portugal*

Abstract

This work investigates the application of a Local Search (LS) enhanced Genetic Programming (GP) algorithm to the control scheme's design task. The combination of LS and GP aims to produce an interpretable control law as similar as possible to the optimal control scheme reference. Inclusive Genetic Programming (IGP), a GP heuristic capable of promoting and maintaining the population diversity, is chosen as the GP algorithm since it proved successful on the considered task. IGP is enhanced with the Operators Gradient Descent (OPGD) approach, which consists of embedding learnable parameters into the GP individuals. These parameters are optimized during and after the evolutionary process. Moreover, the OPGD approach is combined with the adjoint state method to evaluate the gradient of the objective function. The original OPGD was formulated by relying on the backpropagation technique for the gradient's evaluation, which is impractical in an optimization problem involving a dynamical system because of scalability and numerical errors. On the other hand, the adjoint method allows for overcoming this issue. Two experiments are formulated to test the proposed approach, named Operator Gradient Descent - Inclusive Genetic Programming (OPGD-IGP): the design of a Proportional-Derivative (PD) control law for a harmonic oscillator and the design of a Linear Quadratic Regulator (LQR) control law for

an inverted pendulum on a cart. OPGD-IGP proved successful in both experiments, being capable of autonomously designing an interpretable control law similar to the optimal ones, both in terms of shape and control gains.

7 *Keywords:* Genetic Programming, Gradient Descent, Adjoint State
8 Method, Control

9 1. Introduction

10 Genetic Programming (GP) [1] is a powerful algorithm to evolve com-
11 puter programs, represented as trees, by iteratively selecting, recombining,
12 and mutating a population of candidate solutions. Thanks to this symbolic
13 representation, GP generates solutions that, differently to the ones achieved
14 with other artificial intelligence (AI) techniques, may be interpreted (i.e.,
15 when the GP trees present a limited number of nodes) by domain experts.
16 Nevertheless, the search performed by GP operators (crossover and mutation)
17 is solely syntactic. Thus, there is no explicit parameter optimization during
18 the evolutionary process. This can lead to evident drawbacks, as pointed out
19 by Castelli et al. [2]. For instance, let us consider the scenario where the
20 evolutionary search led to an individual with the following syntax $K(x) =$
21 $x + \sin(x)$, while the optimal solution is $K^*(x) = 3.3x + 1.003 \sin(0.0001x)$.
22 Since there is no explicit parameters optimization, the solution $K(x)$ might
23 be easily lost during the selection phase, leading to a very inefficient process.
24 Including a Local Search (LS) routine in traditional GP has proven to be
25 an effective method to overcome this limitation [2, 3, 4]. The advantages of
26 embedding a gradient-based approach as an LS method in the evolutionary
27 GP flow have emerged clearly in tasks such as symbolic regression [5] and
28 image classification [6].

29 The objective of this study is to demonstrate that this combination can
30 also play a critical role in control applications and Intelligent Control (IC)
31 tasks. Intelligent control refers to the use of Artificial Intelligence (AI) to
32 design control systems capable of adapting and learning from their environment.
33 This approach allows for more robust and flexible control systems that can
34 adapt to changing conditions and improve their performance over time [7, 8].
35 The approach proposed in this work cannot be classified as IC since the
36 controller design phase is performed offline. Nonetheless, where GP offers
37 a compelling option for generating comprehensible control laws [9, 10], thus

38 providing a major benefit with respect to other AI alternatives, such as Neu-
 39 ral Networks (NNs). Interpretability is especially relevant in control appli-
 40 cations, where knowledge of the control equation is essential for evaluating
 41 systems' reliability and behaviour. For example, in linear systems, the knowl-
 42 edge of the control law expression is used to build the closed-loop transfer
 43 function of the whole system [11]. This is then used to perform stability
 44 analysis. Moreover, in the context of AI applied to control systems, hav-
 45 ing an interpretable control law helps increase the trust towards AI-based
 46 control systems, making the connection between input and output explicit
 47 [12]. The use of GP for control system generation has been previously doc-
 48 umented in the literature [13, 14]. Yet, it remains relatively infrequent, and
 49 to the best of the authors' knowledge, this is the first application of a com-
 50 bination of GP and gradient-descent-based LS to the task of control system
 51 design. Specifically, the method developed by Pietropolli et al. [5], named
 52 Operators Gradient Descent (OPGD), has been used in this work, consider-
 53 ing the promising results reported in its previous application [5]. The idea
 54 underpinning OPGD is simple and yet effective: learnable parameters are
 55 embedded in GP programs, and the standard GP evolutionary approach is
 56 combined with a gradient-based refinement of the individuals. In this study,
 57 the method has been adequately modified to deal with control problems. In
 58 the original OPGD, the backpropagation technique was employed to evaluate
 59 the gradient of the fitness function w.r.t. the GP parameters. The backprop-
 60 agation is impractical to use in control problems since an implicit dependency
 61 between the state and control variables appears in the chain of derivatives.
 62 The implicit dependency is caused by the absence of the analytic expression
 63 of the states, which results in the impossibility of evaluating, symbolically,
 64 the partial derivatives of the states w.r.t. the control variables. Automatic
 65 differentiation could be used to avoid the symbolic evaluation of the afore-
 66 mentioned partial derivative, but it would require performing the backprop-
 67 agation through the ODE solver, which leads to a high memory cost and
 68 introduces additional numerical error [15]. A different approach that would
 69 avoid the backpropagation through the solver and that scales efficiently to
 70 large problems is the adjoint state method [16] applied in this work. To test
 71 the suitability of this OPGD variant, two control problems were chosen: a
 72 harmonic oscillator controlled by a Proportional-Derivative (PD) control law
 73 and an inverted pendulum on a cart controlled by a Linear Quadratic Reg-
 74 ulator (LQR) control law. Experimental results confirm the validity of the
 75 proposed algorithm: the produced control laws are well-performing in terms

of fitness and control task, and the integration of a local search strategy leads to a substantial improvement in both the desired control structure and the associated parameters compared with others GP-based approaches without any LS mechanism and a feedforward NN.

This paper is structured as follows: Section 2 reviews previous work on combining LS strategies in GP and GP applied to design a control law. Section 3 describes the overall framework introduced in this work, comprising a detailed description of the OPGD technique, IGP algorithm, and adjoints state method, and how they are combined.

Subsequently, Section 4 describes the two control problems chosen to test the ability of the GP-based algorithm, and Section 5 discusses the results of the experimental campaign. Finally, Section 6 summarizes the main contribution achieved in this study and provides directions for future works.

2. Related Works

This section reviews existing work related to the method developed in this study. In particular, Section 2.1 outlines contributions concerning the combination of GP and local search strategies and then presents recent papers in which gradient-descend-based algorithms have been coupled with the GP evolutionary process. Subsequently, Section 2.2 briefly discusses different approaches for the design of control laws, highlighting the reason for using a GP-based approach in this paper.

2.1. GP with local search and gradient-based algorithms

A refinement process consists of embedding a LS strategy in the evolutionary process. In particular, the additional LS operator considers one or more individuals and searches for the local optima near them. These techniques are a simple type of memetic algorithm [17], which exploits the fact that Evolutionary Algorithms (EAs) can explore large areas of the search space while local optimizers improve solutions gradually and steadily. Their complementary strengths have inspired a lot of novel research in recent years [18, 19, 3, 4, 20].

While several works linking EAs and LS can be found in the literature, the ones that focus on the combination of GP and LS constitute a limited subset [18]. In Eskridge and Hougen [21], authors introduced the LS directly on the GP crossover operator, named memetic crossover, that allows individuals to imitate the observed success of others. Later, in Wang et al. [22], authors

111 proposed a new GP algorithm with local search strategies, named Memetic
 112 Genetic Programming (MGP), for dealing with classification problems. An-
 113 other example can be found in Muñoz et al. [3], where authors proposed a
 114 sequential GP memetic structure with Lamarckian inheritance. In this case,
 115 two LS methods have been combined: a greedy pruning algorithm and least
 116 squares parameter estimation.

117 Focusing on the combination of GP and gradient-descent-based algo-
 118 rithms, examples can be found in the literature [23, 6, 5, 24]. Nevertheless,
 119 existing contributions deal with a specific task or focus on particular compo-
 120 nents of the evolutionary search. For instance, in Topchy et al. [23], the au-
 121 thors complemented a genetic search for tree-like programs at the population
 122 level with terminal values optimization via gradient descent at the individ-
 123 ual level. Experimental results show that tuning random constants, besides
 124 improving fitness results, requires minimal computational overhead. Zhang
 125 and Smart [6] applied a gradient descent algorithm to the numeric paramete-
 126 r terminals in each individual program for object classification problems.
 127 Two methods (an online gradient descent scheme and an offline gradient de-
 128 scent scheme) are developed and compared with the basic GP. Experimental
 129 results demonstrated that introducing this kind of LS outperforms standard
 130 GP in terms of classification accuracy and training time. Another application
 131 dealing with constant values optimization can be found in Graff et al. [25],
 132 where authors considered the problem of time series forecasting, specifically
 133 wind speed time series.

134 The first example of the inclusion of weight parameters at the internal
 135 nodes level is described in the work of Smart and Zhang [26]. Here, a pa-
 136 rameter called the inclusion factor is assigned to each node, and a gradient
 137 descent search is applied to the inclusion factors. This method obtained
 138 promising results, but the experimental study only considered classification
 139 tasks. Moreover, the GP system was evaluated using an unusually narrow
 140 function set (only sum and multiplication), which is an unrealistic config-
 141 uration. Later, in Kommenda et al. [27], a gradient-based non-linear least
 142 squares optimization algorithm, i.e., Levenberg Marquardt, is used for ad-
 143 justing constant values in symbolic expression trees during their evolution.
 144 Additionally, artificial nodes are inserted in the symbolic expression tree to
 145 account for the linear scaling terms.

146 In Trujillo et al. [19], a Lamarckian memetic GP incorporates LS strategy
 147 to refine GP individuals. A simple parametrization for GP trees, where
 148 the same functions share the same coefficients, is proposed with different

149 heuristic methods to determine which individuals should be subject to the LS.
 150 More recently, in Harrison et al. [28], authors investigate how gradient-based
 151 techniques can optimize coefficients in symbolic regression tasks. Lastly, in
 152 Pietropolli et al. [5], the authors proposed embedding learnable parameters in
 153 GP programs and combining the standard GP evolutionary approach with a
 154 gradient-based refinement of the individuals employing the Adam optimizer.
 155 Two different algorithms (that differ in how these parameters are shared in
 156 the expression operators) are proposed and subsequently tested on real-world
 157 problems, demonstrating proficiency in significantly outperforming plain GP.

158 Due to its simplicity, this GP tree embedding can be easily integrated
 159 into other GP approaches, as done in this work. Specifically, in our study,
 160 this LS strategy has been applied to a variant of GP, namely the IGP de-
 161 veloped by Marchetti and Minisci [29]. IGP was specifically developed for
 162 control problems, where it is used to design a control law. Its peculiarity is
 163 the capability to promote and maintain population diversity during the evo-
 164 lutionary process. Moreover, it proved superior to a standard GP algorithm,
 165 both on control law design and regression tasks. A more detailed description
 166 of the IGP is provided in Subsection 3.2.

167 2.2. GP for the design of control laws

168 The use of GP to design a control law is not novel in the literature. Koza
 169 himself [30] pointed out the capability of GP to automatically design human
 170 competitive control laws. Other recent examples can be found in the work of
 171 Verdier and Mazo, Jr. [31], where GP is employed to automatically produce
 172 a control Lyapunov function and the modes of a switched state feedback con-
 173 troller. In Lapa et al. [32], the authors applied GP to evolve a Proportional
 174 Integral Derivative (PID) based controller resistant to noise. To this end,
 175 they used a Genetic Algorithm (GA) to optimize the parameters in a GP
 176 control law. In contrast, the method described in this manuscript employs
 177 a gradient-based LS technique, thus resulting in a lower computational time
 178 compared to a GA algorithm. Another interesting example of GP based
 179 Symbolic Regression (SR) used to design a controller is presented in the
 180 work of Danai and La Cava [33], where the authors applied a variant of GP,
 181 the Epigenetic Linear Genetic Programming (ELGP), to produce the models
 182 describing the open-loop input for a desired plant output. This inverse solu-
 183 tion approach allows for avoiding the time-consuming closed-loop controller
 184 evaluation by performing algebraic evaluations.

185 In this paper, GP was chosen since it can produce interpretable models.
186 However, other AI techniques can be used to design control laws, for exam-
187 ple, NNs. Plenty of research exists on this topic. Some early applications
188 are described in the book of Irwin et al. [34], while a recent survey of NN
189 control systems applied to aerospace vehicles can be found in [35]. A work
190 closely related to our study is AI Pontryagin of Böttcher et al. [36]. AI
191 Pontryagin is an NN-based control framework capable of designing optimal
192 control laws. Nonetheless, the models produced by this method are not in-
193 terpretable. Because of the lack of interpretability produced by NNs or other
194 AI algorithms, we did not perform a thorough comparison of our approach
195 with these techniques. In fact, the objective of this study is to produce in-
196 terpretable control laws that resemble the optimal control law of reference,
197 both in terms of shape and parameters.

198 **3. Parametrized GP with Adjoint State Method**

199 This Section contains a detailed explanation of the building blocks form-
200 ing the OPGD-IGP algorithm introduced in this work. The OPGD and IGP
201 algorithms are described along with a detailed discussion on gradient eval-
202 uation techniques, justifying the choice of the adjoint state method. This
203 Section concludes with a schematic summary of the overall framework.

204 *3.1. Parameterized Genetic Programming*

205 One of the main strengths of GP is the possibility of interpreting the
206 solutions that it generates. Nevertheless, the search performed by a GP
207 algorithm only relies on syntactic operations, such as crossover and muta-
208 tion, to improve the quality of the individuals. In fact, standard GP does
209 not adjust the (implicit) parameters of the given expression. To overcome
210 this problem, different possibilities for integrating a LS algorithm in the GP
211 routine have been proposed in recent years. In this work, we enhance the
212 expressive capability of GP individuals by adding learnable parameters on
213 their operators, as proposed in [5]. The resulting GP individuals are inter-
214 pretable as parametric functions, which can be optimized via gradient-based
215 techniques. A canonical GP individual can be represented as a tree where
216 all the edge connections between nodes take a constant value of 1. Yet, the
217 possibility of modifying those values leads to a large spectrum of possible
218 solutions. An example follows.

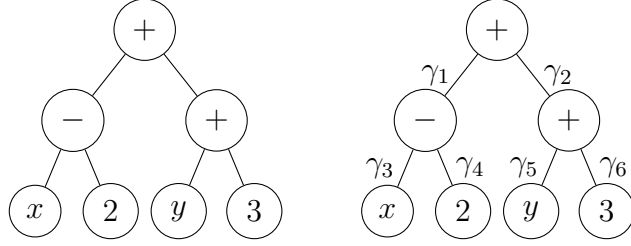


Figure 1: Depiction of the plain and parametrized GP tree.

Figure 1 shows, on the left, a canonical GP individual encoding the expression in Equation 1:

$$(x - 2) + (y + 3) \quad (1)$$

On the right, the same GP individual is enriched with the addition of the parameters γ_i over all the edge connections and encodes the expression in Equation 2:

$$\gamma_1 \cdot (\gamma_3 \cdot x - \gamma_4 \cdot 2) + \gamma_2 \cdot (\gamma_5 \cdot y + \gamma_6 \cdot 3) \quad (2)$$

Equation 2 would correspond to Equation 1 if all the weights γ_i were set to 1.

The Operators Gradient Descent (OPGD) [5] is used, which assigns a different set of weights to each instance of the GP operators, leading to a total number of parameters equal to the number of nodes in the tree. Moreover, to fully exploit the LS potential, a gradient-based optimization of the parameters is performed both during and after the evolutionary process. When the optimizer is used after each generation, it is applied to the whole population of individuals. On the other hand, when applied at the end of the evolutionary process, it is used solely on the best individual of the population obtained. This optimization can be performed using different optimization algorithms, both local and global. The algorithms employed in this work are Adam [37] (during the evolutionary process) and the Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm [38] at the end of it. Adam was chosen to achieve faster optimizations during the evolutionary process, while BFGS is preferred at the end of the evolution to better improve the partial results obtained during the evolution. The overall evolutionary process enhanced with the OPGD approach is summarized in Algorithm 1.

In the original OPGD approach, Adam was used in combination with the backpropagation technique to evaluate the gradient, and this resulted

Algorithm 1 Pseudocode of evolutionary process with OPGD approach

```
1: Initialize population
2: Store best individual
3: for  $i = 1 \rightarrow N_{generations}$  do
4:   for  $j = 1 \rightarrow N_{individuals}$  do
5:     Insert learnable parameters in j-th individual
6:     Perform optimization of j-th individual, using Adam with a learn-
       ing rate  $\alpha$  for  $n_{opt}$  steps.
7:     Assign the lowest fitness found at the previous step to the j-th
       individual
8:     Remove learnable parameters from j-th individual
9:   end for
10:  Perform crossover and mutation to generate offspring
11:  Evaluate fitness of offspring repeating lines 4 to 9.
12:  Apply selection to generate new population
13:  Update best individual
14: end for
15: Insert learnable parameters in the best individual from all generations
16: Optimize the best individual with BFGS
```

244 in a fast optimization process. Nonetheless, as explained in Section 3.3, a
245 different approach to evaluate the gradient becomes necessary when dealing
246 with control problems.

247 *3.2. Inclusive Genetic Programming*

248 OPGD can be applied to any GP formulation. In this work, it is ap-
249 plied to the Inclusive Genetic Programming (IGP) introduced in [29]. The
250 resulting method is referred to as OPGD-IGP. IGP was chosen because it
251 was developed specifically for control applications and showed superior per-
252 formance than standard GP thanks to its ability to promote and maintain
253 the genotypic population's diversity.

254 Greater genotypic diversity means that bigger individuals are not dis-
255 carded by the bloat control operators but considered during the crossover and
256 mutation operations, and only the selection is performed to favor smaller in-
257 dividuals. The genotypic material of these bigger individuals may capture the
258 nonlinearities of the studied dynamical system better than smaller individu-
259 als, and it is thus an essential piece of information. IGP applies a modified

version of the $\mu + \lambda$ evolutionary strategy, the Inclusive $\mu + \lambda$ summarized in Algorithm 2. The core operations are the niches' creation mechanism, the Inclusive Crossover and Mutation, and the Inclusive Tournament. A detailed description of each of these operations is given in [29]. Briefly, a newly created population is subdivided into niches, which act as containers for individuals with a determined size. The maximum and minimum size that a niche can contain is defined by linearly dividing the interval between the maximum and minimum size of the individuals in the population by $n + 1$, where n is the number of niches. The Inclusive Crossover and Mutation consist of applying crossover and mutation by selecting individuals from different niches, and the Inclusive Tournament is a Double Tournament sequentially applied to each niche. Using these operations, a wider distribution of individuals' lengths is considered, and genotypic information is not lost during the evolutionary process due to bloat control operators.

Algorithm 2 Pseudocode of Inclusive $\mu + \lambda$ evolutionary strategy

```

1: Perform population initialization
2: Best individual all-time  $\leftarrow$  Best individual initial population
3: for  $i = 1 \rightarrow N_{generations}$  do
4:   Generate  $n$  niches from the current population
5:   Perform Inclusive Crossover and Inclusive Mutation to generate  $\lambda$ 
     offspring from  $\mu$  parents
6:   Apply Inclusive Tournament to select  $\mu$  individuals from a starting
     population of  $\mu$  parents +  $\lambda$  offspring
7:   if Fitness Best individual  $population_i <$  Fitness Best individual all
     time then
8:     Best individual all-time  $\leftarrow$  Best individual  $population_i$ 
9:   end if
10: end for

```

3.3. Gradient Evaluation Techniques

Gradient-based search algorithms perform the gradient evaluation during the optimization process. The gradient can be evaluated with different approaches, the most common of which is the finite differences approach, which gives a numerical approximation of the gradient at a computational cost proportional to the problem's dimensionality (i.e., the number of optimization

variables). For limiting the computational cost, other approaches are employed, such as the backpropagation algorithm [39] often used to optimize a NN's parameters. Backpropagation efficiently computes chain of partial derivatives of the entire NN model [40], leading to a straightforward evaluation of the gradient. However, as explained in the following, backpropagation becomes impractical in control problems. The adjoint state method [41] is chosen as an alternative to evaluate the gradient in the OPGD algorithm applied to a control problem. An additional benefit of this technique is that it can scale efficiently to problems with a high number of parameters.

The rest of this Subsection contains a brief demonstration of why the backpropagation approach is impractical in control problems and a description of the adjoint state method.

3.3.1. Backpropagation

The backpropagation algorithm is an efficient approach to evaluating derivatives by leveraging the chain rule. In classical regression problems, it is possible to build the entire chain of derivatives to express the gradient of an objective function J with respect to the optimization variables γ . As an example, a regression problem is considered, and GP is used to create a regression model. The considered GP individual is a function of the selected features and a set of parameters γ , as described in Subsection 3.1. The goal is to find the optimal set of parameters γ such that a fitness function J is minimized. J can be evaluated as the Mean Square Error (MSE) between the output of the GP model $\hat{z}$ and the desired output z , as $J = \frac{1}{n} \sum_{i=1}^n (z_i - \hat{z}_i)^2$, where n is the number of samples in the dataset. Using the chain rule, the gradient of J w.r.t. γ can be computed as shown in Equation 3.

$$\frac{\partial J}{\partial \gamma} = \frac{\partial J}{\partial \hat{z}} \frac{\partial \hat{z}}{\partial \gamma} \quad (3)$$

Since $\hat{z}$ (produced by the GP algorithm) is expressed in symbolic form, the partial derivatives in Equation 3 can be evaluated analytically. Nonetheless, when considering a control problem, i.e. a dynamical system, an implicit dependency between the state variables and the control variables appears. As an example, a control problem is considered where $\mathbf{u}$ is the vector of control variables and $\mathbf{y}$ is the vector of state variables. The dynamical system is defined by Equation 4, where GP is used to design the control law.

$$\dot{\mathbf{y}} = \mathbf{f}(\hat{\mathbf{y}}(t), \mathbf{u}(t)) = \mathbf{f}(\hat{\mathbf{y}}(t), \mathbf{u}_{GP}(\hat{\mathbf{y}}(t), \gamma)) \quad (4)$$

312 The GP model is expressed as a function of the states $\hat{\mathbf{y}}$ and parameters
 313 $\boldsymbol{\gamma}$. The goal is to track a desired trajectory $\mathbf{y}$ by finding the optimal set of
 314 parameters $\boldsymbol{\gamma}$ that minimizes the error between the obtained trajectory $\hat{\mathbf{y}}$ and
 315 the desired trajectory $\mathbf{y}$. By using the chain rule, Equation 5 is obtained.

$$\frac{\partial J}{\partial \boldsymbol{\gamma}} = \frac{\partial J}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \boldsymbol{\gamma}} = \frac{\partial J}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial u} \frac{\partial u}{\partial \boldsymbol{\gamma}} \quad (5)$$

316 In Equation 5, the term $\frac{\partial \hat{\mathbf{y}}}{\partial u}$ represents an implicit dependency since the
 317 analytical expression of the states is not known. Thus, this partial derivative
 318 cannot be evaluated analytically but only using numerical methods.

319 The straightforward approach would be to use the finite differences tech-
 320 nique, which results in a high computational cost. In fact, if the dynamical
 321 system is composed of d differential equations and p optimizable parameters,
 322 the cost of applying the finite differences is $\mathcal{O}(d(p+1))$, i.e., $p+1$ Ordinary
 323 Differential Equations (ODE) propagations at the cost of d differential equa-
 324 tions. Another approach is the continuous local sensitivity analysis, which
 325 scales proportionally with the number of optimization parameters and leads
 326 to a cost of $\mathcal{O}(dp)$ [42]. A last alternative is the adjoint state method used
 327 in this work. This algorithm computes one forward pass of the ODE system
 328 composed of d differential equations and one backward pass of the adjoint
 329 dynamical system composed of p differential equations, leading to a compu-
 330 tational cost of $\mathcal{O}(d+p)$. Moreover, the derivatives involved in the adjoint
 331 method can be computed symbolically, leading to lower computational errors
 332 than other numerical methods [43].

333 3.3.2. Adjoint State Method

334 The adjoint state method has its roots in optimal control theory. It
 335 allows the evaluation of the gradient by defining the Lagrangian of the cost
 336 functional and the related adjoint variable. The steps to perform the gradient
 337 evaluation with the adjoint state method are described in the following. The
 338 derivation of the adjoint state method equations presented in the remaining
 339 part of this section is taken from [44] and adapted to our work.

340 Consider the dynamical system in the form of Equation 6, where $\mathbf{y}$ are the
 341 state variables, $\mathbf{u}$ the control variables, $\boldsymbol{\gamma}$ the optimization variables, and $\mathbf{f}$ is
 342 a nonlinear mapping describing the initial value problem with $\mathbf{y}(t=0) = \mathbf{y}_0$
 343 as initial conditions.

$$\dot{\mathbf{y}} = \mathbf{f}(\mathbf{y}(t), \mathbf{u}(\mathbf{y}(t), \boldsymbol{\gamma})) \quad (6)$$

344 The goal of the optimization process is to minimize a functional in the
 345 form of Equation 7. To do so, the gradient of J with respect to γ is sought,
 346 as illustrated in Equation 8

$$J(\mathbf{y}, \gamma) = \int_0^T g dt + h(T) \quad (7)$$

$$\frac{dJ}{d\gamma} = \int_0^T \frac{dg}{d\gamma} dt + \frac{dh}{d\gamma}(T) = \int_0^T \left(\frac{\partial g}{\partial \gamma} + \frac{\partial g}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \gamma} \right) dt + \frac{dh}{d\gamma}(T) \quad (8)$$

347 The term $\frac{\partial \mathbf{y}}{\partial \gamma}$ in Equation 8 cannot be computed analytically due to the
 348 implicit relation between $\mathbf{y}$ and γ . To overcome this issue, the optimization
 349 can be framed as an equality-constrained minimization problem by intro-
 350 ducing the Lagrangian of the function, as in Equation 9, with the associate
 351 adjoint variable ν .

$$\mathcal{L}(\mathbf{y}, \gamma, \nu) = J(\mathbf{y}, \gamma) + \int_0^T \nu(t)^T \left(\mathbf{f} - \frac{d\mathbf{y}}{dt} \right) dt \quad (9)$$

352 The gradient of the Lagrangian is then computed as in Equation 10. The
 353 last term in the integral in Equation 10 can be integrated by part resulting
 354 in Equation 11

$$\frac{d\mathcal{L}}{d\gamma} = \int_0^T \left(\frac{\partial g}{\partial \gamma} + \nu(t)^T \frac{\partial \mathbf{f}}{\partial \gamma} + \left(\frac{\partial g}{\partial \mathbf{y}} + \nu(t)^T \frac{\partial \mathbf{f}}{\partial \mathbf{y}} \right) \frac{d\mathbf{y}}{d\gamma} - \nu(t)^T \frac{d}{dt} \frac{d\mathbf{y}}{d\gamma} \right) dt + \frac{dh}{d\gamma}(T) \quad (10)$$

$$\begin{aligned} \frac{d\mathcal{L}}{d\gamma} = & \int_0^T \left(\frac{\partial g}{\partial \gamma} + \nu(t)^T \frac{\partial \mathbf{f}}{\partial \gamma} + \left(\frac{\partial g}{\partial \mathbf{y}} + \nu(t)^T \frac{\partial \mathbf{f}}{\partial \mathbf{y}} + \left(\frac{d\nu}{dt} \right)^T \right) \frac{d\mathbf{y}}{d\gamma} \right) dt + \\ & + \nu(0)^T \frac{d\mathbf{y}}{d\gamma}(0) - \nu(T)^T \frac{d\mathbf{y}}{d\gamma}(T) + \frac{dh}{d\gamma}(T) \end{aligned} \quad (11)$$

355 Since the optimization problem was rewritten as an equality-constrained
 356 optimization, the goal is to set the second term in Equation 9 to zero, there-
 357 fore resulting in $\mathcal{L}(\mathbf{y}, \gamma, \nu) = J(\mathbf{y}, \gamma)$. According to this, it can be stated that
 358 the gradient of the Lagrangian in Equation 11 corresponds to the gradient
 359 of the functional ∇J_γ .

360 By setting some of the elements in Equation 11 to zero, we can use it
 361 to evaluate the gradient of the functional. The resulting set of equations is
 362 summarized in Equation 12.

$$\frac{dJ}{d\boldsymbol{\gamma}} = \int_0^T \left(\frac{\partial g}{\partial \boldsymbol{\gamma}} + \boldsymbol{\nu}(t)^T \frac{\partial \mathbf{f}}{\partial \boldsymbol{\gamma}} \right) dt \quad (12a)$$

$$\frac{d\boldsymbol{\nu}}{dt} = - \left(\frac{\partial \mathbf{f}}{\partial \mathbf{y}} \right)^T \boldsymbol{\nu}(t) - \left(\frac{\partial g}{\partial \mathbf{y}} \right)^T \quad (12b)$$

$$\boldsymbol{\nu}(t = T) = \frac{dh}{d\mathbf{y}}(T) \quad (12c)$$

363 Employing the notation introduced in [44], and summarized in Equation
 364 13, Equation 12 can be simplified as Equation 14

$$\mathbf{A} = \frac{\partial \mathbf{f}}{\partial \mathbf{y}}, \mathbf{B} = \frac{\partial \mathbf{f}}{\partial \boldsymbol{\gamma}}, \boldsymbol{\eta} = \frac{\partial h}{\partial \mathbf{y}}, \boldsymbol{\phi} = \frac{\partial g}{\partial \mathbf{y}}, \boldsymbol{\psi} = \frac{\partial g}{\partial \boldsymbol{\gamma}} \quad (13)$$

$$\frac{dJ}{d\boldsymbol{\gamma}} = \int_0^T \left(\boldsymbol{\psi} + \boldsymbol{\nu}(t)^T \mathbf{B} \right) dt \quad (14a)$$

$$\frac{d\boldsymbol{\nu}}{dt} = -\mathbf{A}^T \boldsymbol{\nu}(t) - \boldsymbol{\phi}^T \quad (14b)$$

$$\boldsymbol{\nu}(t = T) = \boldsymbol{\eta}(T) \quad (14c)$$

365 The overall process of evaluating the gradient using the adjoint state
 366 method can be summarized in three steps:

- 367 1. Forward propagation of the dynamic system in Equation 4.
- 368 2. Backward propagation of the adjoint system in Equation 14b evaluating
 369 the initial conditions with Equation 14c. This propagation is performed
 370 from $t = T$ to $t = 0$.
- 371 3. Evaluate the gradient with Equation 14a and the fitness function with
 372 Equation 7

373 The adjoint state method can be applied effortlessly to optimize a GP
 374 control law. The parametrized GP laws enter the dynamical system as in
 375 Equation 4, where $\boldsymbol{\gamma}$ are the parameters to be optimized or the optimization
 376 variables in the optimization problem. The GP law $\mathbf{u}_{GP}$ can be built using

only differentiable functions, resulting in a differentiable equation that can be inserted explicitly in the equation of motion. Subsequently, the partial derivatives in Equation 13 can be evaluated symbolically.

From this point onward, the acronym OPGD will be used to refer to the OPGD with the gradient evaluation performed using the adjoint state method.

3.4. OPGD-IGP Framework Summary

Figure 2 presents the frameworks of plain OPGD and OPGD-IGP. The latter represents the novel approach introduced in this work. The novelties introduced in this work are highlighted in Figure 2 by the colored boxes and are the following: 1) the GP algorithm to which OPGD is applied; 2) the target application or data source; 3) the approach used to evaluate the gradient; 4) the optimizer used to optimize the best individual found during the evolutionary process. The original OPGD was used to enhance a standard GP algorithm, while in this work, it was applied to IGP as highlighted by the *GP algorithm* box. The second difference lies in the data passed to OPGD. In OPGD-IGP, the data originated through the interaction with a dynamical system, while in the original OPGD, a static dataset was used (*Data source* box). Because of this different data source, a different approach to evaluate the gradient is employed (*Gradient evaluation approach* box), as explained in the previous subsections. Lastly, to optimize the best individual found during the evolutionary process, OPGD-IGP relies on the BFGS optimization algorithm, while in the standard OPGD, Adam was employed (*Optimizer* box).

4. Experimental Study - Test Cases

Two control problems are chosen to test the ability of the OPGD-IGP to design a control law automatically, both in terms of shape and parameters: a harmonic oscillator controlled by a PD control scheme and an inverted pendulum controlled by an LQR control scheme.

4.1. Harmonic Oscillator

The formulation of the harmonic oscillator is the one defined in [44] and described by the nonlinear ODE system in Equation 15. The state variables are the position x and the speed v . Thus, $\mathbf{y} = [x, v]$. u is the control variable.

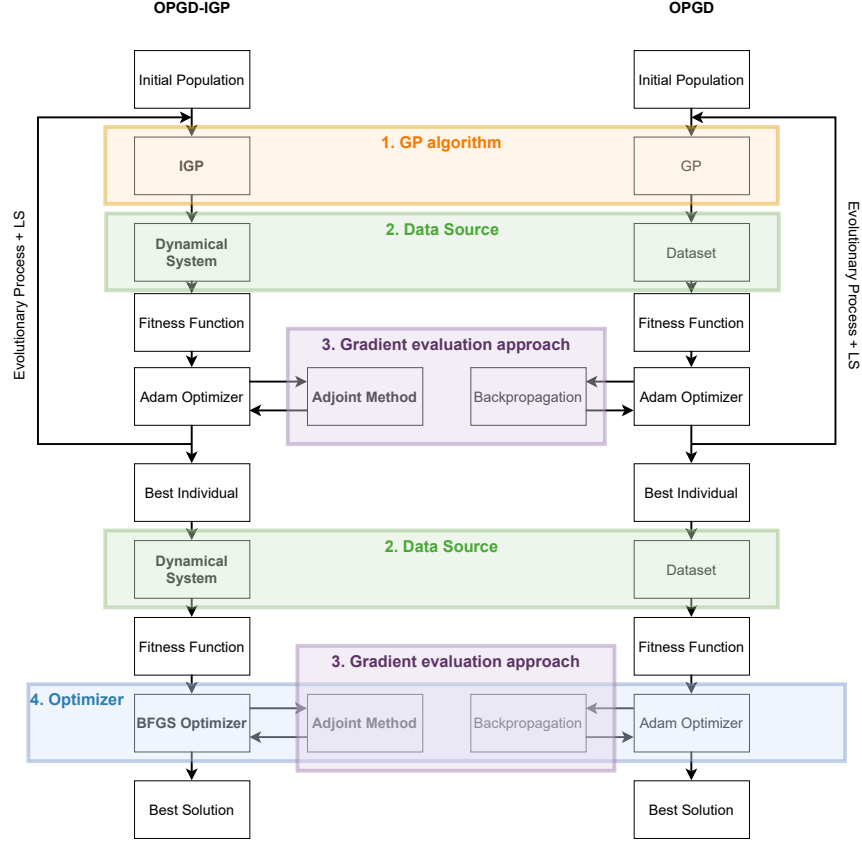


Figure 2: Diagrams of the OPGD-IGP (left) and OPGD (right) workflows.

$$\begin{aligned}
 \dot{x} &= v \\
 \dot{v} &= -\frac{k}{m}x - \frac{c}{m}(ax^2 + b)v + \frac{u}{m}
 \end{aligned} \tag{15}$$

410 The constant parameters used in Equation 15 are reported in Table 1.
 411 The initial conditions were set as $x_0 = 4 \text{ m}$, $v_0 = 0 \text{ m/s}$, $t_0 = 0 \text{ s}$, while the
 412 desired final conditions are $x_f = 0 \text{ m}$, $v_f = 0 \text{ m/s}$, $u_f = 0 \text{ N}$, $t_f = 10 \text{ s}$.

413 The control scheme designed for this test case is a PD control scheme
 414 that receives as input the tracking errors on the position e_x and speed e_v .
 415 The methodology used to obtain the proportional and derivative gains is
 416 described in [44]. Equation 16 illustrates the final control law used as a
 417 reference.

Parameter	Value	Description
m	1 kg	Mass
k	2 kg/s ²	Spring stiffness
a	1 m ⁻²	First damper coefficient
b	-1	Second damper coefficient
c	0.3 kg/s	Third damper coefficient

Table 1: Harmonic oscillator parameters

$$u = -1.753e_x - 3.010e_v \quad (16)$$

4.2. Inverted Pendulum on a Cart

The formulation of the inverted pendulum was taken from Brunton and Kutz [45] and described by the nonlinear ODE system in Equation 17. The states variables are the position x , speed v , angular position θ and angular speed ω , therefore $\mathbf{y} = [x, v, \theta, \omega]$. u is the control variable.

$$\begin{aligned} \dot{x} &= v \\ \dot{v} &= \frac{-m^2 L^2 g \cos(\theta) \sin(\theta) + mL^2 (mL\omega^2 \sin(\theta)) + mLu^2}{mL^2 (M + m(1 - \cos(\theta)^2))} \\ \dot{\theta} &= \omega \\ \dot{\omega} &= \frac{(m + M)mgL \sin(\theta) - mL \cos(\theta) (mL\omega^2 \sin(\theta)) - mL \cos(\theta)u}{mL^2 (M + m(1 - \cos(\theta)^2))} \end{aligned} \quad (17)$$

The constant parameters used in Equation 17 are described in Table 2. The initial conditions were set as $x_0 = -1$ m, $v_0 = 0$ m/s, $\theta_0 = \pi + 0.1$ rad, $\omega_0 = 0$ rad/s, $t_0 = 0$ s, and the desired final conditions are $x_f = 1$ m, $v_f = 0$ m/s, $\theta_f = \pi$ rad, $\omega_f = 0$ rad/s, $u_f = 0$ N, $t_f = 10$ s.

The same LQR design process described in [45] was used, with $\mathbf{Q}$ set as a 4×4 identity matrix and $R = 1$.

The reference control law for the LQR scheme is displayed in Equation 18, where the input variables are the errors on the states.

$$u = -\mathbf{K}\mathbf{e} = 1e_x + 1.419e_v - 8.131e_\theta - 1.223e_\omega \quad (18)$$

The parameters used to design the LQR controller were chosen to have the LQR gains close to 1. This is necessary to have a good outcome from the

Parameter	Value	Description
M	0.1 <i>kg</i>	Cart mass
m	0.02 <i>kg</i>	Pendulum mass
L	0.1 <i>m</i>	Pendulum length
g	-9.8 <i>m/s</i> ⁻²	Gravitational acceleration

Table 2: Inverted pendulum parameters

433 optimization process: because a local optimization scheme was employed,
434 the choice of the initial condition influences the optimization process. Since
435 no prior information is available on the initial value of the GP parameters,
436 these were initialized as 1. Therefore, the optimization process converges if
437 the desired value is close to 1 as well. Different optimization approaches can
438 be used to deal also with larger gain values. Nonetheless, it is not the aim of
439 this work to explore different optimization algorithms.

440 5. Experimental Results

441 To test the proposed methodology, Standard Genetic Programming (SGP),
442 IGP, OPGD-IGP, and a feedforward NN were compared. The control laws'
443 parameters for the two considered control schemes of reference were obtained
444 through an optimization process. Therefore, the goal of the experimental
445 campaign is to use the aforementioned algorithms to design a well-performing
446 control law by solving the same optimization problem as those considered in
447 the references. The similarity between the obtained control laws and the ref-
448 erence ones, both in terms of shape and parameters, is considered to assess
449 the success of the experiments.

450 On the other hand, the NN does not produce interpretable models and
451 is only used as a reference to understand how OPGD-IGP compares against
452 a different and more established approach. The NN is trained in two ways:
453 1) with a dataset produced using the optimal control laws of reference -
454 this experiment is meant to discover the smallest configuration necessary to
455 learn the desired model; 2) training the NN in-the-loop as done with the
456 OPGD-IGP. This last training method is summarized in Algorithm 3

457 A discussion of the outcome of each training method is provided at the
458 end of Subsections 5.3 and 5.4 for the oscillator and pendulum test cases
459 respectively. A description of the dataset and training results of the former
460 approach is provided in Appendix A. This result represents the capability

Algorithm 3 Pseudocode of the training process with NN in-the-loop

```
1: Create NN model
2: Extract the NN weights and store them in the vector of optimization
   variables  $\mathbf{p}$ 
3: Start optimization process
4: while Termination criteria is not met do
5:   Insert the updated weights from  $\mathbf{p}$  into the NN
6:   Propagate the ODE system using the NN as controller
7:   Evaluate the objective function according to the obtained trajectory
8:   Evaluate the gradient with the adjoint state method
9:   Update the  $\mathbf{p}$  vector with the optimizer routine
10: end while
```

of the algorithms to autonomously design an optimal control law for the
chosen controller architecture, both in terms of shape and parameters. For
each the SGP, IGP and OPGD-IGP algorithms, 30 independent runs were
performed to obtain a statistical sample. The Adam optimizer in OPGD-IGP
considered a learning rate of 0.01 and 5 optimization steps, respectively α
and n_{opt} in Algorithm 1, during the evolutionary process. At the end of
the evolutionary process, the best individual is optimized using the BFGS
algorithm implemented in the Python library Scipy [46]. A fitness function
precision threshold of 10^{-6} was used as termination criterion for the BFGS
algorithm. BFGS with these settings was used to train the NN in-the-loop
as well. The developed code will be available at <https://github.com/strath-ace/smart-ml>.

5.1. GP settings

The common settings of OPGD-IGP, IGP, and SGP for the two test
cases are listed in Table 3. IGP and SGP use two ephemeral constants, while
OPGD-IGP does not consider ephemeral constants. This choice is motivated
by the fact that OPGD-IGP should be able to find the correct parameters
autonomously. On the other hand, ephemeral constants are necessary to
allow IGP and SGP to evolve parametric control laws.

5.2. Fitness Function

For the two test cases, the fitness function described in Equation 7 was
used. The functions g and h in Equation 7 are set as quadratic functions, as
described in Equations 19 and 20,

	Oscillator	Pendulum
Population Size	300 individuals	
Maximum Generations	300	
Stopping criteria	Reaching maximum number of generations	
Crossover probability	0.2 $\rightarrow$ 0.65	
Mutation probability	0.7 $\rightarrow$ 0.25	
Evolutionary strategy	Inclusive $\mu + \lambda$	
μ	Population size	
λ	Population Size $\times$ 1.2	
Limit Height	10	
Limit Size	15	30
Selection Mechanism	Inclusive Tournament	
Double Tournament fitness size	2	
Double Tournament parsimony size	1.2	
Tree creation mechanism	Ramped half and half	
Mutation mechanisms	Uniform (50%), Shrink (5%), Insertion (25%), Mutate Ephemeral (20%)	
Crossover mechanism	One point crossover	
Primitives Set	+, -, $\times$	

Table 3: SGP, IGP and OPGD-IGP settings for both test cases.

$$g = \frac{1}{2}(\mathbf{e}_y^T \mathbf{Q}_g \mathbf{e}_y + \mathbf{e}_u^T \mathbf{Q}_u \mathbf{e}_u) \quad (19)$$

$$h = \frac{1}{2} \mathbf{e}_y^T \mathbf{Q}_h \mathbf{e}_y \quad (20)$$

where $\mathbf{e}_y$ is the vector of the tracking errors on the state variables, and $\mathbf{e}_u$ is the vector of the tracking errors on the control variables. $\mathbf{Q}_g$, $\mathbf{Q}_u$, $\mathbf{Q}_h$ are diagonal matrices used to weight the different contributions to the fitness function. These functions are used to minimize the tracking errors on the states and control variables. Using Equation 7, the integral of g is evaluated, leading to the minimization of both the states and controls tracking errors on the whole trajectory. h is used to evaluate the tracking error on the final position. In this work, also the tracking for the complete trajectory is performed against the desired final conditions. Therefore, each reference trajectory can be imagined as a constant line at the desired value of the considered state or control variable.

5.3. Harmonic Oscillator

For this test case, the fitness function's parameters were set as follows: $\mathbf{e}_y = [e_x, e_v]$, $\mathbf{e}_u = e_u$, $\mathbf{Q}_g = \text{diag}([5, 5])$, $\mathbf{Q}_u = 1$, $\mathbf{Q}_h = \text{diag}([1, 1])$. The tracking errors are evaluated as the difference between the current state and control variables and the desired final values listed in Subsection 4.1. Using the reference control law, a reference fitness equal to $J = 56.152$ was obtained by applying Equation 7. The fitness was evaluated by propagating the dynamical system using the Runge-Kutta 4 scheme with an integration step of 0.05 seconds.

The obtained results are presented from Figure 3 to Figure 7 and in Appendix B. In the following Figures, *NN Data* refers to the NN trained on the dataset, while *NN Loop* refers to the NN optimized in the control loop. The smallest NN architecture capable of capturing the optimal control law behaviour is composed of one hidden layer with one neuron. More details are given in Appendix A. The same configuration is trained in-the-loop to compare the effect of a different training approach.

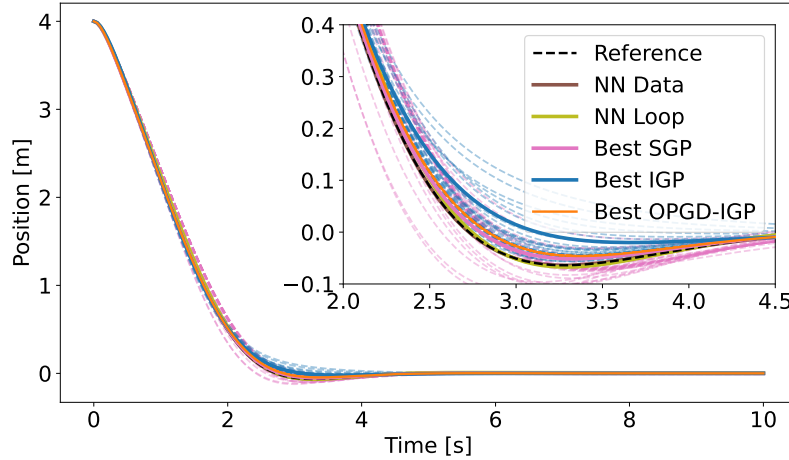


Figure 3: Harmonic oscillator's position x trajectories obtained using SGP, IGP, OPGD-IGP, and the NN models.

Figures 3 and 4 depict the state trajectories, while Figure 5 shows the control force trajectories. In these plots, the reference trajectory, obtained via the reference control law, is depicted as a dashed black line. The inset in each plot highlights the distribution of the obtained trajectories. As can be seen in Figures 3, 4 and 5, all the tested algorithms evolve well-performing control

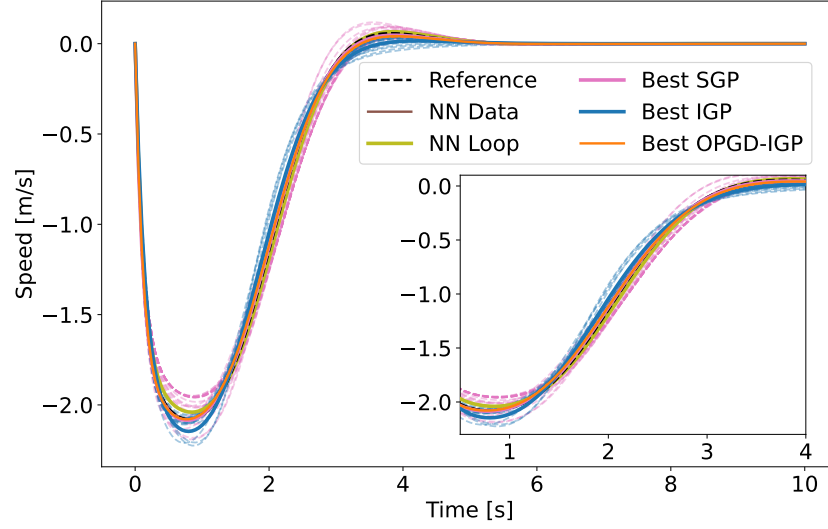


Figure 4: Harmonic oscillator's speed v trajectories obtained using SGP, IGP, OPGD-IGP, and the NN models.

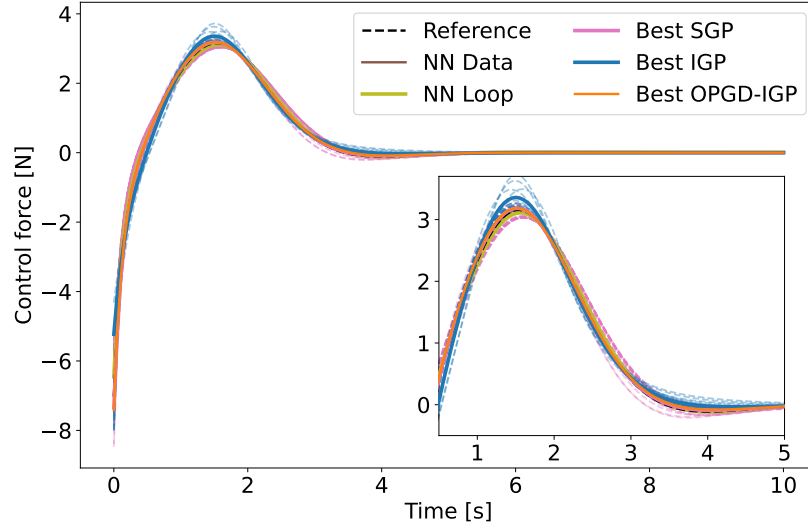


Figure 5: Harmonic oscillator's control action u trajectories obtained using SGP, IGP, OPGD-IGP, and the NN models.

516 laws, capable of generating a behaviour close to the reference one. Among the
 517 GP algorithms, it can be seen how SGP is the least consistent, with many of
 518 the produced trajectories straying from the reference. When considering IGP

519 and OPGD-IGP, the magnified sections show that IGP produces control laws
 520 that result in a wider range of behaviours, while the trajectories produced
 521 with OPGD-IGP are all overlapped, meaning that they always converge to
 522 the same mathematical model.

523 Figures 6 and 7 depict statistical analyses of the obtained fitness val-
 524 ues. Specifically, Figure 6 shows the best individual’s fitness evolution. It
 525 is possible to observe that OPGD-IGP can reach the final solution in fewer
 526 generations (~ 65 generations) with respect to IGP (> 100 generations),
 527 while SGP results are worse than the other two GP-based algorithms. Fig-
 528 ure 7 displays the fitness values obtained in the simulations performed with
 529 the GP algorithms and the fitness values obtained with both NN’s training
 530 approaches. The fitness of the NN trained in-the-loop comes naturally from
 531 the optimization process, while the fitness of the NN trained on the data is
 532 obtained by propagating a trajectory with the trained model and evaluating
 533 the fitness as described in Subsection 5.2.

534 Looking at Figure 7, it can be seen how OPGD-IGP always converges
 535 to the same individual, while IGP tends to produce different control laws
 536 with different performance and, as observed also from Figures 3 to 5, SGP
 537 is the least consistent performer among the GP algorithms. Moreover, these
 538 boxplots show that IGP can achieve a lower fitness value than OPGD-IGP.
 539 This is likely due to the random mutation applied to the ephemeral constants.
 540 This mechanism, absent in OPGD-IGP, allows for a greater exploration of
 541 the search space in contrast to the exploitation fostered by the use of LS.
 542 Regarding the NNs, it can be seen how the two training approaches lead to
 543 slightly different results. In fact, the fitness obtained with the NN trained on
 544 the data matches almost exactly the reference fitness, while the one trained
 545 in-the-loop shows a fitness worse than IGP and OPGD-IGP. This suggests
 546 that a network with more parameters is required to eventually improve the
 547 results with the train in-the-loop approach.

548 The complete list of models produced by the GP algorithms can be found
 549 in Appendix B. As one can observe, IGP and SGP produce a variety of mod-
 550 els, while OPGD-IGP always converges to the same combination of control
 551 law shape and parameters, thus confirming the ability of OPGD-IGP to au-
 552 tonomously produce the desired control law for a dynamical system in terms
 553 of shape and parameters. Table 4 lists the reference control law and the
 554 most frequent OPGD-IGP control law. The difference between the reference
 555 and the obtained optimal parameters is caused by the different optimization
 556 algorithms used in this work and in [44].

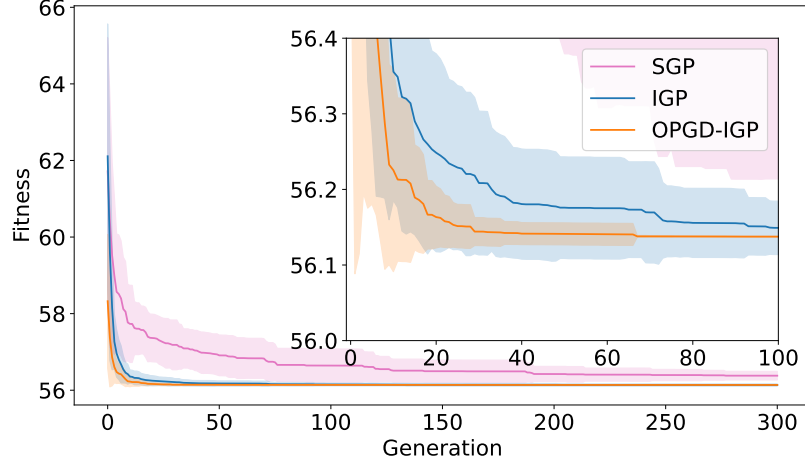


Figure 6: Fitness evolution of the SGP, IGP, and OPGD-IGP algorithms for the harmonic oscillator case. The solid lines represent the mean, while the shaded areas depict the error bands, i.e. standard deviations, for both algorithms.

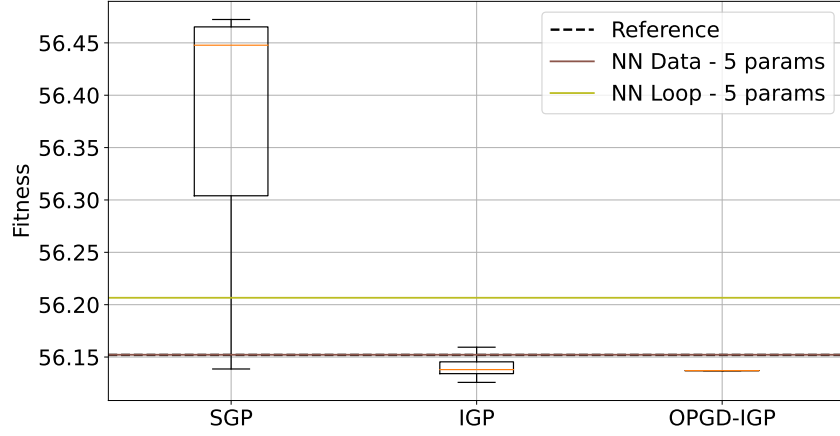


Figure 7: Fitness of the best-performing individual for the SGP, IGP, OPGD-IGP, and NN algorithms models for the harmonic oscillator case. For the GP-based algorithms, 30 simulations were considered.

5.4. Inverted Pendulum on a Cart

For this test case, the fitness function's parameters were set as follows:
 $\mathbf{e}_y = [e_x, e_v, e_\theta, e_\omega]$, $\mathbf{e}_u = e_u$, $\mathbf{Q}_g = \text{diag}([5, 5, 5, 5])$, $\mathbf{Q}_u = 1$, $\mathbf{Q}_h = \text{diag}([1, 1, 1, 1])$.
The tracking errors are evaluated between the current and the desired final

	Control Law
Reference	$-1.753e_x - 3.010e_v$
OPGD-IGP	$-1.854e_x - 3.158e_v$

Table 4: Reference control law and most frequent model output by the OPGD-IGP for the harmonic oscillator test case.

values reported in Subsection 4.2. The optimization problem is structured in a slightly different way than the reference. The same fitness function is used, but different plant models are employed.

As for the previous test case, in the following Figures, *NN Data* refers to the NN trained on the dataset, while *NN Loop* refers to the NN optimized in the control loop. Again, the smallest NN architecture capable of capturing the optimal control law behaviour consists of one hidden layer with one neuron. More details are given in Appendix A. As for the oscillator case, the same configuration is trained in-the-loop to assess the effect of a different training approach.

Using the reference control law, a fitness value $J = 16.264$ is obtained. The fitness is evaluated by applying Equation 7 after propagating the dynamical system using the Runge-Kutta 4 scheme with an integration step of 0.01 seconds. The integration step was reduced to 0.005 seconds to perform the training of the NN in-the-loop (more details at the end of this subsection.)

Figures 8 to 12 depict the trajectories obtained by propagating the dynamical system using the control laws evolved by the SGP, IGP, and OPGD-IGP algorithms on the 30 simulations performed and by the best-performing NN architectures (obtained using both training approaches). As for the previous test case, the continuous lines represent the best solution, while the dim dashed lines depict the other ones. The black dashed line represents the reference trajectory.

Figures 13 and 14 show the statistical analysis of the fitness results obtained. As for the oscillator test case, Figure 13 highlights the faster convergence of OPGD-IGP compared to IGP. OPGD-IGP can reach the minimum fitness in ~ 40 generations while IGP requires more than 100 generations. As for the previous test case, SGP performs worse than the other two GP algorithms.

Figure 14 displays the fitness obtained with the tested algorithms. The boxplots for the GP algorithms are created considering the best fitness achieved

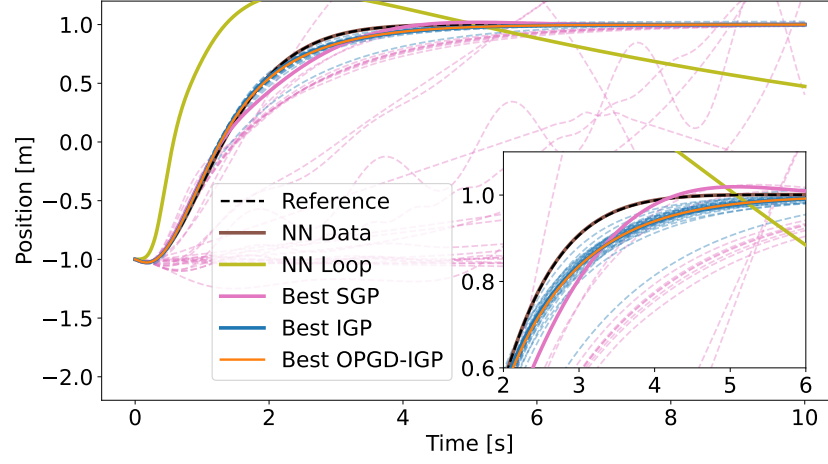


Figure 8: Trajectories of the pendulum's position x obtained using SGP, IGP, OPGD-IGP and the NN models

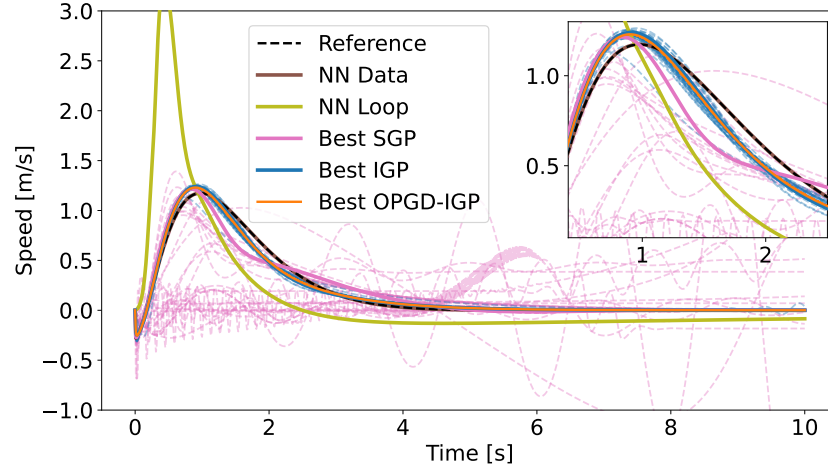


Figure 9: Trajectories of the pendulum's speed v obtained using using SGP, IGP, OPGD-IGP and the NN models.

591 in each of the 30 simulations. OPGD-IGP converges to similar individuals
592 and also reaches a lower fitness compared to the IGP algorithm, while the
593 SGP produced individuals with worse performance than the other GP algo-
594 rithms. Regarding the NN results, the training from data led to an almost
595 perfect match with the optimal solution, while the training in-the-loop led

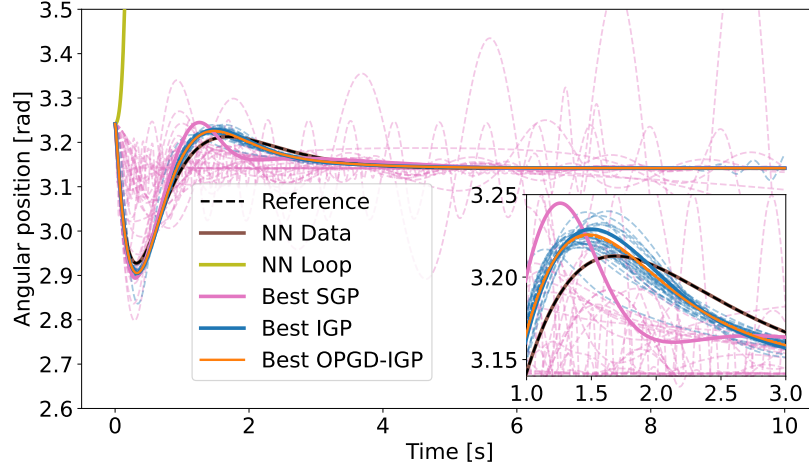


Figure 10: Trajectories of the pendulum's angular position θ obtained using using SGP, IGP, OPGD-IGP and the NN models

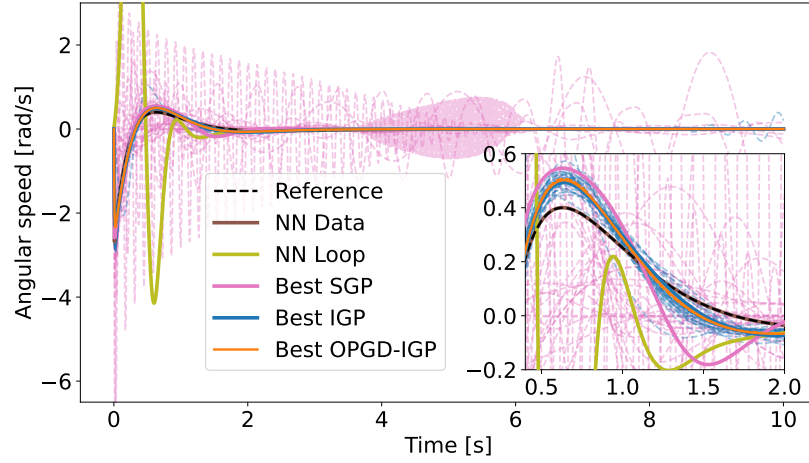


Figure 11: Trajectories of the pendulum's angular speed ω obtained using using SGP, IGP, OPGD-IGP and the NN models

596 to poor results. These results are discussed in Subsection 5.5.

597 The complete list of the models produced by the GP algorithms is listed
 598 in Appendix B. These results show that OPGD-IGP can often (21/30 simu-
 599 lations) converge to individuals with the same shape and similar parameters
 600 to the reference one. On the other hand, IGP produces only one model
 601 (simulation 5) with the same shape and similar parameters as the reference.

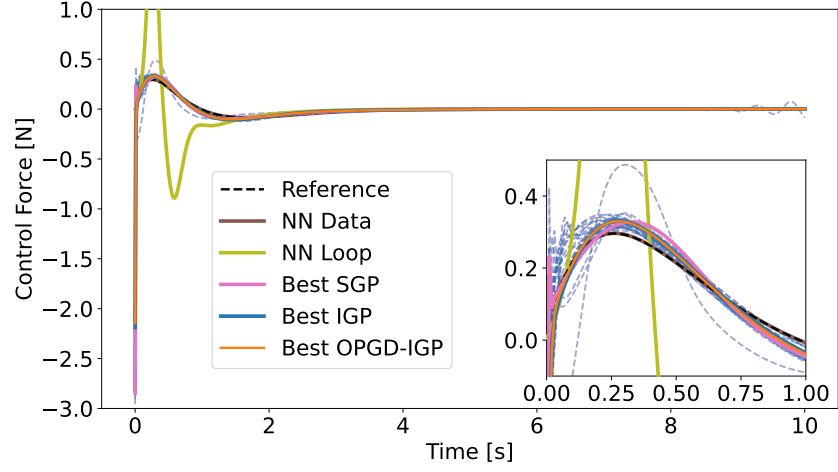


Figure 12: Trajectories of the pendulum's control force u obtained using SGP, IGP, OPGD-IGP and the NN models

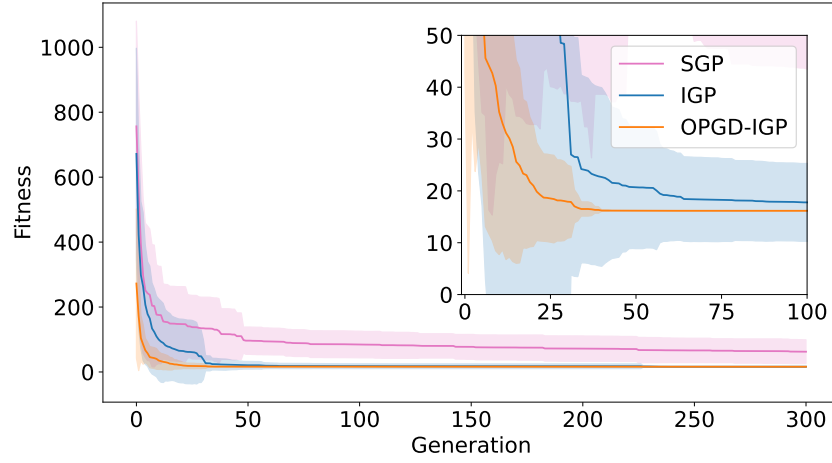


Figure 13: Fitness evolution of the SGP, IGP and OPGD-IGP algorithms for the pendulum case. The solid lines represent the mean, while the shaded areas show the error bands, i.e. standard deviations.

SGP is capable of finding more models with the appropriate shape than IGP. However, the parameters are far from their optimal values (e.g., simulations 7, 10, 11, 24, 27), and the overall result is a set of models that perform worse than those found by IGP.

Table 5 lists the reference control law and the most frequent model ob-

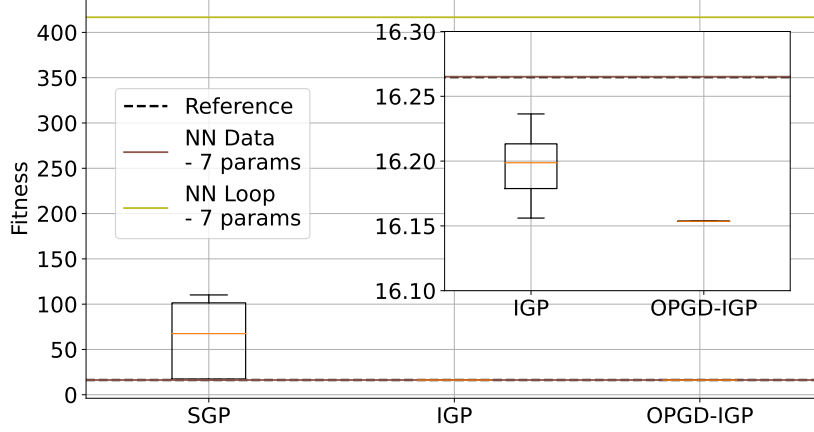


Figure 14: Fitness of the best-performing individual for the SGP, IGP, OPGD-IGP and NN models for the pendulum case. For the GP algorithms, 30 simulations were considered.

607 tained by the OPGD-IGP method.

Control Law	
Reference	$1e_x + 1.419e_v - 8.131e_\theta - 1.223e_\omega$
OPGD-IGP	$0.781e_x + 1.161e_v - 5.842e_\theta - 0.952e_\omega$

Table 5: Reference control law and most frequent model output by OPGD-IGP for the inverted pendulum test case.

608 The difference in the parameters' values is caused by the difference in the
609 plant's models used to obtain them and the employed optimization schemes.
610 The LQR gains are evaluated by solving the continuous-time algebraic Ric-
611 cati equation using the linearized plant models. On the other hand, in
612 OPGD-IGP, the parameters are optimized with a numerical scheme, and
613 the complete nonlinear models are considered. This result is particularly
614 interesting since it showcases how OPGD-IGP can be applied to a fully non-
615 linear model and still produces a control law close to the optimal one. This
616 approach would allow designing an optimal control law even for complex
617 systems that cannot be linearized or without resorting to linearization tech-
618 niques that can cause a loss of information.

5.5. Discussion on Neural Network Results

Two different training methodologies were tested for the NN: 1) training with a dataset generated from the reference optimal control law and 2) training within the control loop. The first approach primarily serves to determine the minimal configuration capable of learning the reference optimal control law. In fact, this approach cannot be directly compared with the OPGD-IGP since the latter learns how to control a system by interacting with it, while the NN trained on pre-existing data lacks knowledge of the system to be controlled. Furthermore, if the control law is available and used to produce the dataset, creating a regression model on those data becomes superfluous.

The objective of OPGD-IGP is to generate an interpretable control law, similar to the optimal one both in shapes and parameters, by solving the same optimization problem used to find the reference control law. Consequently, this study aims to demonstrate that the OPGD-IGP can autonomously find an interpretable and optimal control law solely by interacting with the controlled system knowing only the high-level goal, i.e., the objective function of the optimization problem, and with no prior knowledge of the reference control law itself. That is why the NN is also trained in-the-loop, i.e., in the same training setting used by OPGD-IGP. The smallest configuration found after training on the data is considered since it proves that the NN has enough parameters to learn the desired control law. Thus, it should also be able to do it when trained in-the-loop. While this is true in the oscillator case, where the two training approaches lead to similar results, it is not true in the pendulum case. This discrepancy can be traced back to the greater nonlinearity of the pendulum’s ODE system compared to the oscillator one. This translates into a greater sensitivity to the control input and makes the training in-the-loop a complex local optimization problem. It was observed that the NN’s weight initialization plays a crucial role in this. In fact, by varying the initialization, the results vary significantly. Few initialization approaches were tested, but none led to satisfactory results.

The training in-the-loop required lowering the integration step from 0.01 to 0.005 to stabilize the ODE propagation, which is another proof of the greater instability of the pendulum’s ODE system and its sensitivity to the control input. On the other hand, OPGD-IGP can successfully find a good model because, during the evolutionary process, it learns to discard those solutions that lead to a failure of the ODE system’s propagation. The results of the NN trained in-the-loop could improve by increasing its complexity and performing a thorough study of several initialization techniques. However,

657 this would result in a non-interpretable model, hence straying from the scope
658 of this work.

659 6. Conclusions

660 This work applies OPGD-IGP, an IGP algorithm enhanced with a gradient-
661 based LS strategy proposed by some of the authors in a previous work for
662 automatically designing a control law for a desired plant.

663 OPGD was designed for dealing with regression problems and leverages
664 the backpropagation technique to evaluate the gradient of the fitness func-
665 tion w.r.t the GP parameters. The backpropagation is impractical to use in
666 control problems due to the implicit dependency of the state variables on the
667 control variables. To overcome this issue, this study used the adjoint state
668 method. The adjoint state method is a powerful mathematical approach that
669 allows the evaluation of the gradient of an optimization problem involving
670 a dynamical system with minimal computational effort and numerical errors
671 compared to other techniques.

672 The proposed method was tested on two test cases: a harmonic oscillator
673 controlled by a PD control law and an inverted pendulum on a cart controlled
674 by a LQR control law. The objective of the experiments was to test the
675 OPGD-IGP’s capability to automatically design a control law similar, in
676 terms of parameters and shape, to the reference one by leveraging the intra-
677 evolution LS optimization. To understand the importance of the LS applied
678 to GP, the performances achieved by OPGD-IGP have been compared with
679 the ones achieved by IGP (a GP variant that does not involve any LS step),
680 a Standard GP (SGP) without any LS step, and a feedforward NN. The NN
681 was trained with two different approaches. First, it was trained on the data
682 produced using the reference control laws. This training was performed to
683 find the minimal NN topology necessary to capture the optimal control law
684 behaviour. Secondly, the NN with the minimal topology was trained in-the-
685 loop, i.e., by interacting with the dynamical system as done by OPGD-IGP.
686 The NN trained with this last approach is the one to consider when comparing
687 NN with OPGD-IGP.

688 IGP and OPGD-IGP proved capable of performing the desired task, being
689 able to produce a well-behaving control law for all the performed simulations
690 with a good resemblance of shape and parameters with the reference con-
691 trol law. In particular, in the oscillator problem, IGP evolved 20/30 control
692 laws with the same shape as the reference and similar parameters. On the

693 other hand, OPGD-IGP achieved the desired shape and parameters in 30/30
694 simulations. Regarding the pendulum, IGP produced the desired shape and
695 parameters only in 1/30 simulation while OPGD-IGP did it in 21/30 simu-
696 lations.

697 Different performances were observed for SGP and the NN trained in-
698 the-loop. Both performed well when applied to the oscillator test case. SGP
699 produced a control law with the desired shape on 28/30 simulations. How-
700 ever, despite obtaining more models than IGP with the same shape as the
701 reference, the resulting behaviors were more varied and less consistent than
702 those produced by IGP. The NN trained in-the-loop was capable of con-
703 trolling the system successfully, resulting in a fitness comparable to the one
704 achieved by the GP-based algorithms. On the other hand, both SGP and
705 NN trained in-the-loop showed poor performance when applied to the pen-
706 dulum test case. This can be explained by the greater nonlinearity of the
707 considered system, resulting in a more complex optimization problem that
708 appeared extremely sensitive to the provided initial conditions.

709 These results confirm that GP is a valid alternative to classical approaches
710 for automatically designing a control law. In particular, the use of LS com-
711 bined with the GP evolutionary process led to inferring the optimal shape
712 and parameters of the desired control law, in contrast with a GP approach
713 not enhanced with an LS, where the control laws are different from each other
714 and also different from the ground-truth. Moreover, comparing OPGD-IGP
715 and SGP results on the oscillator case, it can be seen how the SGP can
716 achieve the desired shape almost as often as the OPGD-IGP, although the
717 parameters' values are randomly assigned. On the other hand, using an LS
718 within the evolutionary process allows GP to find both the optimal shape and
719 parameters. Finally, OPGD-IGP showed better performance than a feedfor-
720 ward NN. This result can be explained by the ability of GP to evolve models
721 with different genotypes but with a phenotype close to the reference control
722 law. Thus, GP can compensate for the sensitivity to the initial conditions in
723 the pendulum test case by discarding those models that lead to a failure of
724 the dynamical system propagation.

725 The obtained results have important implications, such as allowing con-
726 trol practitioners to automate the control law design process and explore new
727 control law formulations when dealing with complex nonlinear problems. In
728 fact, the results show that an optimal control law can be produced automat-
729 ically also by considering the full nonlinear system.

730 Future research will focus on three directions. First, it would be interest-
731 ing to apply OPGD-IGP online to create an IC system. This would fully ex-
732 ploit the LS phase to adapt to unforeseen disturbances. Second, OPGD-IGP
733 could be applied to more complex systems to automatically develop adaptive
734 control schemes that otherwise would require an extensive design effort from
735 the engineers. Lastly, the comparison between IGP and OPGD-IGP on the
736 oscillator case shed light on the benefits of promoting exploration during the
737 evolutionary process. It would be interesting to analyze the effects of a ran-
738 domized initialization of the learnable parameters during the evolutionary
739 process. This approach could lead to the exploitation of different local min-
740 ima through the LS and allow the discovery of novel and better-performing
741 control schemes.

742 **Acknowledgments**

743 This work was supported by national funds through the FCT (Fundação
744 para a Ciência e a Tecnologia) by the project UIDB/04152/2020-Centro de
745 Investigação em Gestão de Informação – MagIC/NOVA IMS; and by the
746 SPECIES Society through the SPECIES Scholarship 2022.

747 **Appendix A. Neural Networks Training from Data: Settings and** 748 **Results**

749 This appendix contains the settings used to train the NNs from the data
750 and a summary of the training outcome.

751 *Appendix A.1. Dataset*

752 10000 samples were generated using a Latin Hypercube Sampling be-
753 tween $[-2, 2]$ for all the input features. These were then passed to Equations
754 16 and 18 to generate the corresponding output data for the oscillator and
755 pendulum test cases. This way, one dataset for the oscillator case and one
756 dataset for the pendulum case were created. The datasets were then split
757 into train+validation (80%) and test datasets (20%). The train+validation
758 dataset was further split into train (80%) and validation (20%) datasets.

759 *Appendix A.2. Architecture and Settings*

760 For both test cases, a minimal architecture consisting of one hidden layer
761 with one neuron was used. This architecture proved sufficient to learn the

optimal control laws from the data, as reported in Subsection Appendix A.3. Linear activation functions were used for each layer since the target model was a linear one. The weights were initialized with the Glorot uniform initialization, and the biases were initialized as zero. Considering all this, the NN model for the oscillator test case contains five tunable parameters, while the one used in the pendulum case contains seven parameters. The difference lies in the different number of inputs.

Appendix A.3. Training

The training was performed with the Adam optimizer with a learning rate of 0.001 for 100 epochs. The MSE was used as a loss function. The plots of the training and validation losses are depicted in Figure A.15, while the prediction performances on the test data are depicted in Figure A.16.

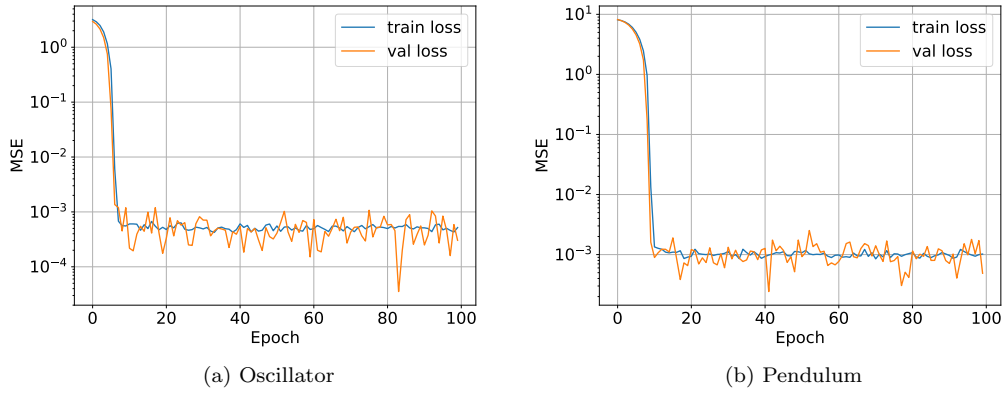


Figure A.15: Train and validation losses for the oscillator and pendulum test cases

The models obtained are listed below and can be compared with Equations 16 and 17.

$$\begin{aligned} u_{NN, Data_{oscillator}} &= -1.966(0.891e_x + 1.530e_v + 0.297) + 0.585 = \\ &= -1.752e_x - 3.009e_v + 0.000262 \end{aligned}$$

$$\begin{aligned} u_{NN, Data_{pendulum}} &= -3.133(-0.319e_x - 0.452e_v + 2.594e_\theta + 0.390e_\omega + \\ &\quad + 0.201) + 0.631 = \\ &= -1.000e_x + 1.418e_v - 8.131e_\theta - 1.222e_\omega - 0.000419 \end{aligned}$$

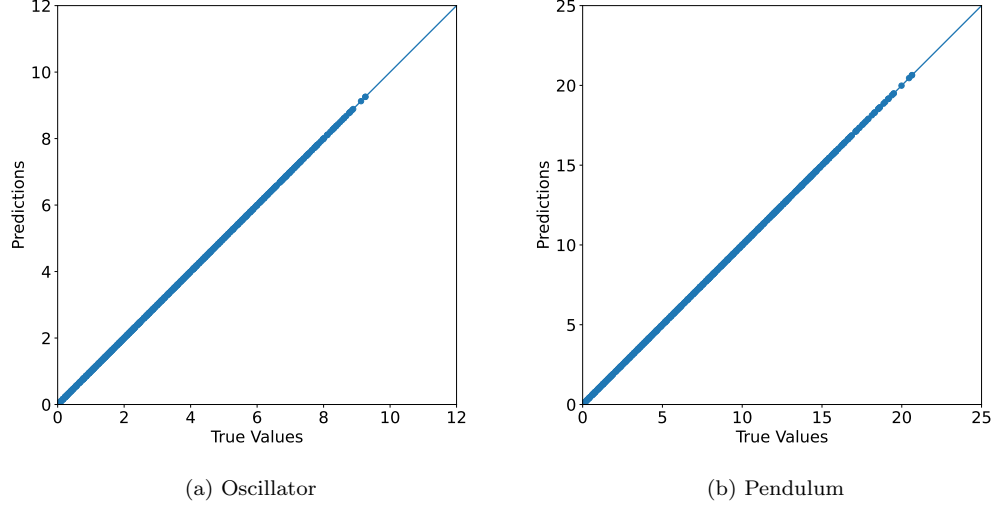


Figure A.16: Comparison of true and predicted output using the test dataset.

776 Appendix B. Produced Control Laws

777 This appendix contains the models produced in all the simulations per-
778 formed with SGP, IGP, and OPGD-IGP. The reported models are obtained
779 by algebraically simplifying the models produced by the GP algorithms.

780 Appendix B.1. Oscillator

781 Appendix B.1.1. SGP

$$\begin{aligned}
u_{SGP_1} &= -1.437e_x - 2.874e_v \\
u_{SGP_2} &= -1.418e_x - 2.836e_v \\
u_{SGP_3} &= -1.456e_x - 2.912e_v \\
u_{SGP_4} &= -1.421e_x - 2.842e_v - 0.011 \\
u_{SGP_5} &= -2.118e_x - 3.566e_v \\
u_{SGP_6} &= -1.421e_x - 2.842e_v \\
u_{SGP_7} &= -1.413e_x - 2.826e_v \\
u_{SGP_8} &= -1.43e_x - 2.86e_v \\
u_{SGP_9} &= -1.424e_x - 2.848e_v
\end{aligned}$$

$$\begin{aligned}
u_{SGP_{10}} &= -1.963e_x - 3.309e_v \\
u_{SGP_{11}} &= -1.407e_x - 2.814e_v \\
u_{SGP_{12}} &= -1.406e_x - 2.751e_v \\
u_{SGP_{13}} &= -1.421e_x - 2.694e_v \\
u_{SGP_{14}} &= -1.674e_x - 2.824e_v \\
u_{SGP_{15}} &= -2e_x - 3.42e_v \\
u_{SGP_{16}} &= -1.418e_x - 2.836e_v \\
u_{SGP_{17}} &= -1.422e_x - 2.844e_v \\
u_{SGP_{18}} &= -1.464e_x - 2.928e_v \\
u_{SGP_{19}} &= -1.506e_x - 2.819e_v \\
u_{SGP_{20}} &= -1.429e_x - 2.858e_v \\
u_{SGP_{21}} &= -1.835e_x - 3.115e_v \\
u_{SGP_{22}} &= -1.806e_x - 2.806e_v \\
u_{SGP_{23}} &= -1.67e_x - 3.082e_v \\
u_{SGP_{24}} &= -1.743e_x - 2.743e_v \\
u_{SGP_{25}} &= -1.421e_x - 2.711e_v \\
u_{SGP_{26}} &= -1.481e_x - 2.718e_v \\
u_{SGP_{27}} &= -1.426e_x - 2.852e_v \\
u_{SGP_{28}} &= -2.097e_x - 3.368e_v \\
u_{SGP_{29}} &= -1.457e_x - 2.914e_v - 0.0393 \\
u_{SGP_{30}} &= -1.419e_x - 2.838e_v
\end{aligned}$$

782 *Appendix B.1.2. IGP*

$$\begin{aligned}
u_{IGP_1} &= -1.775e_x - 3.059e_v \\
u_{IGP_2} &= -1.868e_x - 3.181e_v \\
u_{IGP_3} &= -1.831e_x - 3.123e_v \\
u_{IGP_4} &= -2e_x - 3.324e_v \\
u_{IGP_5} &= e_x(0.378e_v - 1.307) - 3.324e_v \\
u_{IGP_6} &= -1.853e_x - 3.157e_v \\
u_{IGP_7} &= -1.823e_x - 3.142e_v
\end{aligned}$$

$$\begin{aligned}
u_{IGP_8} &= -1.771e_x - 3.108e_v \\
u_{IGP_9} &= -1.875e_x - 3.178e_v \\
u_{IGP_{10}} &= -1.912e_x - 3.234e_v \\
u_{IGP_{11}} &= -2e_x + 0.115(-e_v - 0.208)e_v - 3.517e_v \\
u_{IGP_{12}} &= -1.614e_x + 1.614(0.086e_v - 0.0265)e_x - 3.228e_v \\
u_{IGP_{13}} &= -1.841e_x - 3.138e_v \\
u_{IGP_{14}} &= -1.848e_x - 3.094e_v \\
u_{IGP_{15}} &= -1.871e_x - 3.159e_v \\
u_{IGP_{16}} &= -2e_x - 3.68e_v - 0.139e_v^2 \\
u_{IGP_{17}} &= -1.805e_x - (0.788e_v + 2.840)e_v - 1.805e_v \\
u_{IGP_{18}} &= -1.936e_x - 3.281e_v \\
u_{IGP_{19}} &= (e_v - 0.34(e_v + e_x)^2)(e_x - 3.205) \\
u_{IGP_{20}} &= -1.986e_x - 3.301e_v \\
u_{IGP_{21}} &= -e_x - 2e_v + 0.655(e_v + e_x)(e_x - 4.639) \\
u_{IGP_{22}} &= -1.965e_x - 3.335e_v \\
u_{IGP_{23}} &= -2.384e_x - 3.462e_v - 2.384e_x(-0.093e_v - 0.304) \\
u_{IGP_{24}} &= -1.881e_x - 3.183e_v \\
u_{IGP_{25}} &= -1.841e_x - 3.198e_v \\
u_{IGP_{26}} &= -1.829e_x - 3.134e_v \\
u_{IGP_{27}} &= 1.749e_v(0.00499e_x^2 - 1.749) - 1.749e_x \\
u_{IGP_{28}} &= -1.859e_x - 3.187e_v \\
u_{IGP_{29}} &= -1.892e_x - 3.191e_v \\
u_{IGP_{30}} &= -1.448e_x - 4.804e_v + 0.465e_v(e_x + 2.493)
\end{aligned}$$

783 *Appendix B.1.3. OPGD-IGP*

$$\begin{aligned}
u_{OPGD-IGP_1} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_2} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_3} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_4} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_5} &= -1.854e_x - 3.158e_v
\end{aligned}$$

$$\begin{aligned}
u_{OPGD-IGP_6} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_7} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_8} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_9} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{10}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{11}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{12}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{13}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{14}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{15}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{16}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{17}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{18}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{19}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{20}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{21}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{22}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{23}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{24}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{25}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{26}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{27}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{28}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{29}} &= -1.854e_x - 3.158e_v \\
u_{OPGD-IGP_{30}} &= -1.854e_x - 3.158e_v
\end{aligned}$$

784 *Appendix B.2. Pendulum*

785 *Appendix B.2.1. SGP*

$$\begin{aligned}
u_{SGP_1} &= e_\omega + e_\theta e_x (6.557 - e_\omega) + e_x (e_\omega e_v + e_\omega - e_v) \\
u_{SGP_2} &= -e_\omega^2 - e_\omega - 2e_\theta + e_v
\end{aligned}$$

$$\begin{aligned}
u_{SGP_3} &= -e_\omega - 3.317e_\theta(e_\omega + e_v e_x + 2.495) + e_v + 0.546e_x \\
u_{SGP_4} &= -e_\omega^2 - e_\omega - 2e_\theta + e_v \\
u_{SGP_5} &= e_\theta(-0.48e_\omega e_\theta(2.085e_\omega - e_v + 7.325) - 0.56e_\theta + 2e_v - 2.527) \\
u_{SGP_6} &= -e_\omega^2 + e_v + (e_\omega + 2e_\theta)(e_\theta + e_x) \\
u_{SGP_7} &= -e_\omega - 4.05e_\theta + 2.05e_v + e_x \\
u_{SGP_8} &= e_\theta(e_\omega - 4.203)(e_v + 5.389) - e_v \\
u_{SGP_9} &= e_\theta(e_\omega + e_v + e_x - 13.233) - e_v \\
u_{SGP_{10}} &= -e_\omega - 7.035e_\theta + 2e_v + e_x \\
u_{SGP_{11}} &= -e_\omega - 3e_\theta + e_v - 0.386 \\
u_{SGP_{12}} &= -1.022e_\omega - e_\theta + e_v - 0.204 \\
u_{SGP_{13}} &= -e_\omega - e_\theta(8.154 - e_\omega) + e_v^2 + e_v + e_x \\
u_{SGP_{14}} &= -0.919e_\omega + 0.919e_\theta(e_\omega - 6.911) + 1.838e_v + 0.919e_x \\
u_{SGP_{15}} &= -e_\omega - 3e_\theta + 3e_v \\
u_{SGP_{16}} &= -e_\omega e_v^2 - e_\omega - 7.264e_\theta + e_v(e_v - 0.066) + e_v + e_x \\
u_{SGP_{17}} &= -e_\omega - 2e_\theta + e_v - 0.046e_x - 0.428 \\
u_{SGP_{18}} &= -e_\omega - 17.61e_\theta + e_v^2 + e_x \\
u_{SGP_{19}} &= e_\omega e_v(0.687e_\theta e_x - 0.687e_v + 0.687e_x + 1.025) - 3.209e_\theta \\
u_{SGP_{20}} &= -e_\omega + e_\theta(e_\omega - 5.807) + 2e_v + e_x \\
u_{SGP_{21}} &= -e_\theta - (-e_\omega + 2e_v)(e_\theta + e_x) + (-5.723e_\theta + e_v)(e_v - e_x) \\
u_{SGP_{22}} &= -0.089e_\omega - 5.937e_\theta - e_x^2 + 3.986 \\
u_{SGP_{23}} &= -2.803e_\omega + e_\theta - 1.803e_v + (-e_\theta + e_v)(-8.89e_v - 8.89e_x) - 1.015 \\
u_{SGP_{24}} &= -e_\omega - 7.453e_\theta + 2e_v + e_x \\
u_{SGP_{25}} &= -e_\omega - 5.501e_\theta^2 + e_\theta(e_\omega + e_x) + 2.935e_v \\
u_{SGP_{26}} &= (0.023e_\omega + e_\theta)(4.033e_\omega - 2e_\theta - e_v + e_x + 0.259) \\
u_{SGP_{27}} &= -e_\omega - 5e_\theta + 2e_v + e_x \\
u_{SGP_{28}} &= -e_\omega + 51.050e_\theta(1.045e_v - 1.254) \\
u_{SGP_{29}} &= -2e_\omega e_v - e_\omega - 2e_\theta - e_v^2 + e_v \\
u_{SGP_{30}} &= -e_\omega - 2.926e_\theta(-7.445e_\omega e_\theta e_x - e_\theta e_x + 2.791) + 2e_v + e_x
\end{aligned}$$

$$\begin{aligned}
u_{IGP_1} &= -1.280e_\omega - 8.095e_\theta + 1.560e_v + e_x + e_\theta(e_\omega + e_\theta - e_v - e_x) \\
u_{IGP_2} &= -e_\omega - e_\theta(-9.963e_\omega e_\theta(e_\omega - e_v(e_\theta + 1.5)) + 6.686) - e_\theta + e_v(e_\theta + 1.5) + e_x \\
u_{IGP_3} &= 9.268e_\omega e_\theta^2 - e_\omega - 9.555e_\theta + e_v^2 - e_v(-e_x - 1.686) + e_x \\
u_{IGP_4} &= -e_\omega + e_v + (-6.765e_\theta + e_x)(e_\theta + 0.829)(e_\theta^2(e_\omega - 7.736)(e_\omega + e_v) + 0.829) \\
u_{IGP_5} &= -1.155e_\omega - 6.993e_\theta + 1.47e_v + e_x \\
u_{IGP_6} &= -e_\omega - e_\theta(8.693e_\omega e_\theta(-e_\omega + e_v) - 2.428e_\theta + 6.667) + 1.428e_v + e_x \\
u_{IGP_7} &= -e_\omega + e_\theta(e_v e_x + e_x - 6.787) + e_v(e_v + e_x + 0.664) + e_v + e_x \\
u_{IGP_8} &= -1.16e_\omega + e_\theta(-1.677e_\omega e_v + 5.488e_\theta - 6.197) + 1.345e_v + e_x \\
u_{IGP_9} &= -e_\omega - e_\theta(9.96e_\omega e_\theta - 2e_\theta + e_v - e_x + 4.5) + 1.388e_v + e_x \\
u_{IGP_{10}} &= -e_\omega - 7.709e_\theta + e_v + e_x + (e_x + 2.071)(e_\theta(e_\theta - 6.909) + e_v)(-e_\theta + e_x + 2.071) \\
u_{IGP_{11}} &= -e_\omega - e_\theta(e_\omega + 9.063) + e_v(-e_\theta + e_v + e_x) + 1.715716e_v + e_x \\
u_{IGP_{12}} &= -e_\omega - e_\theta(-e_\omega - 7.203e_\theta + e_v + e_x + 8.802) + e_\theta + e_v(e_\theta + 0.444) + e_v + e_x \\
u_{IGP_{13}} &= (e_\theta + 0.031e_v((e_\omega - 2e_\theta)(e_v + 1.835) - 1.159))(e_\theta e_x - 1.835e_\theta - 2.755) \\
u_{IGP_{14}} &= -e_\omega + e_\theta(0.439e_\omega + 3e_\theta - 5.753) - e_\theta + 1.418e_v + e_x \\
u_{IGP_{15}} &= -e_\omega + e_\theta(e_\omega + e_v e_x - 6.116) - e_\theta + e_v + e_x - (e_\omega - 1.57e_v)(e_\theta + 0.254) \\
u_{IGP_{16}} &= -e_\omega - 8.478e_\theta + e_v(-2e_\theta + e_x) + e_v(e_v - 0.419) + 2e_v + e_x \\
u_{IGP_{17}} &= -e_\omega - e_\theta + 2e_v + e_x + (0.411 - e_\theta)(-0.34e_\omega - 0.660e_\theta - e_v + 6.702) - 2.78 \\
u_{IGP_{18}} &= -e_\omega + e_\theta(e_\omega - e_\theta(-2e_\omega^2 - e_x - 9.545) - 2e_\theta - 6.259) + 1.377e_v + e_x \\
u_{IGP_{19}} &= -1.113e_\omega - 7.299e_\theta - 0.113e_v(-e_\omega - e_\theta - e_x + 3.652) + 2e_v + e_x \\
u_{IGP_{20}} &= -e_\omega + 1.105e_\theta^2 - 0.187e_\theta e_v - 5.726e_\theta + 1.187e_v + 0.813e_x \\
u_{IGP_{21}} &= -e_\omega - 5.08e_\theta + e_v - 0.09072e_x(e_\omega - e_\theta) + 0.676e_x \\
u_{IGP_{22}} &= -e_\omega - e_\theta(-3e_\theta - e_v + 6.92) - e_v(-0.295e_\omega - 0.346) + e_v + e_x \\
u_{IGP_{23}} &= -1.266e_\omega + e_\theta(e_\omega - e_\theta - e_v - 6.694) + 1.532e_v + e_x \\
u_{IGP_{24}} &= -e_\omega - 8.525e_\theta + e_v(-e_\omega e_v - 3e_\theta + e_v^2 + e_x) + 2e_v + e_x \\
u_{IGP_{25}} &= -e_\omega + e_\theta(e_\omega e_\theta(4.593e_\omega - 6.515) + e_\theta - 6.515) - 0.452e_\theta + 1.452e_v + e_x \\
u_{IGP_{26}} &= -0.0241e_\omega(e_v + e_x) - 0.916e_\omega - 5.507e_\theta + 1.083e_v + 0.711e_x \\
u_{IGP_{27}} &= -e_\omega - e_\theta(e_v(-e_\theta(e_\theta + 4.417) + 0.742) + 8.424) + e_v(-e_\theta + e_v + e_x + 0.674) + \\
&\quad + e_v + e_x \\
u_{IGP_{28}} &= -e_\omega + e_\theta(-e_\theta(e_\omega e_\theta + e_\omega)(-e_\omega e_x + e_\omega - 6.744) - 5.207) + e_v + 0.663e_x
\end{aligned}$$

$$\begin{aligned}
u_{IGP_{29}} &= -1.169e_\omega + e_\theta^2 - 6.976e_\theta + 1.413e_v + e_x + 0.0148 \\
u_{IGP_{30}} &= -e_\omega - e_\theta(-e_\omega - 7.074e_\theta + 0.751e_v + e_x + 7.241) + 1.436e_v + e_x
\end{aligned}$$

787 *Appendix B.2.3. OPGD-IGP*

$$\begin{aligned}
u_{OPGD-IGP_1} &= -0.951e_\omega - 0.0274e_\theta(0.0815e_\omega e_\theta + 0.121e_\theta) - 5.839e_\theta + 1.161e_v + 0.780e_x \\
u_{OPGD-IGP_2} &= -0.951e_\omega - 5.839e_\theta + 1.160e_v + 0.780e_x \\
u_{OPGD-IGP_3} &= -0.953e_\omega - 5.844e_\theta + 1.162e_v + 0.781e_x \\
u_{OPGD-IGP_4} &= -0.952e_\omega - 5.840e_\theta + 1.161e_v + 0.780e_x \\
u_{OPGD-IGP_5} &= -0.0172e_\omega e_x - 0.986e_\omega - 5.847e_\theta + 1.162e_v + 0.781e_x \\
u_{OPGD-IGP_6} &= -0.952e_\omega - 5.841e_\theta + 1.161e_v + 0.781e_x \\
u_{OPGD-IGP_7} &= -0.949e_\omega - 5.903e_\theta + 1.174e_v - 0.00900e_x(2.347e_\theta - 1.269e_v) + 0.780e_x \\
u_{OPGD-IGP_8} &= -0.952e_\omega - 5.843e_\theta + 1.162e_v + 0.781e_x \\
u_{OPGD-IGP_9} &= -0.952e_\omega - 5.843e_\theta + 1.162e_v + 0.781e_x \\
u_{OPGD-IGP_{10}} &= -0.952e_\omega - 5.842e_\theta + 1.161e_v + 0.781e_x \\
u_{OPGD-IGP_{11}} &= -0.952e_\omega - 5.842e_\theta + 1.161e_v + 0.781e_x \\
u_{OPGD-IGP_{12}} &= -0.952e_\omega - 5.842e_\theta + 1.161e_v + 0.781e_x \\
u_{OPGD-IGP_{13}} &= -0.954e_\omega + 1.090e_\theta e_v(-1.030e_\omega e_\theta + 0.954e_\theta) + \\
&\quad - 5.806e_\theta + 1.158e_v + 0.783e_x \\
u_{OPGD-IGP_{14}} &= -0.952e_\omega - 5.843e_\theta + 1.162e_v + 0.781e_x \\
u_{OPGD-IGP_{15}} &= -0.952e_\omega - 5.839e_\theta + 1.161e_v + 0.780e_x \\
u_{OPGD-IGP_{16}} &= -0.953e_\omega - 0.010e_\theta e_x(0.998e_\theta - 0.994e_v) - 5.835e_\theta + 1.162e_v + 0.782e_x \\
u_{OPGD-IGP_{17}} &= -0.952e_\omega - 5.843e_\theta + 1.162e_v + 0.781e_x \\
u_{OPGD-IGP_{18}} &= -0.952e_\omega - 5.844e_\theta + 1.162e_v + 0.781e_x \\
u_{OPGD-IGP_{19}} &= -0.953e_\omega - 5.844e_\theta + 1.162e_v + 0.781e_x \\
u_{OPGD-IGP_{20}} &= -0.952e_\omega + 0.0854e_\theta^2 - 5.834e_\theta + 1.161e_v + 0.781e_x \\
u_{OPGD-IGP_{21}} &= -0.952e_\omega - 5.844e_\theta + 1.162e_v + 0.781e_x \\
u_{OPGD-IGP_{22}} &= -0.951e_\omega + 0.00449e_\theta(0.999e_\omega - 1.999e_\theta - 0.999e_v) + \\
&\quad - 5.835e_\theta + 1.160e_v + 0.780e_x \\
u_{OPGD-IGP_{23}} &= -0.952e_\omega - 5.843e_\theta + 1.161e_v + 0.781e_x \\
u_{OPGD-IGP_{24}} &= -0.952e_\omega - 5.842e_\theta + 1.161e_v + 0.781e_x
\end{aligned}$$

$$\begin{aligned}
u_{OPGD-IGP_{25}} &= -0.952e_\omega - 5.842e_\theta + 1.161e_v + 0.781e_x \\
u_{OPGD-IGP_{26}} &= -0.952e_\omega - 5.843e_\theta + 1.162e_v + 0.781e_x \\
u_{OPGD-IGP_{27}} &= -0.952e_\omega - 5.842e_\theta + 1.161e_v + 0.781e_x \\
u_{OPGD-IGP_{28}} &= -0.950e_\omega - 5.862e_\theta + 1.153e_v - 0.00907e_x^2 + 0.7631e_x \\
u_{OPGD-IGP_{29}} &= -0.952e_\omega - 5.840e_\theta + 1.161e_v + 0.780e_x \\
u_{OPGD-IGP_{30}} &= -0.952e_\omega + 0.0264e_\theta(0.999e_\theta - 0.999e_v(1.000e_\omega - 0.999e_v)) \\
&\quad - 5.852e_\theta + 1.162e_v + 0.782e_x
\end{aligned}$$

References

- [1] J. R. Koza, Genetic programming as a means for programming computers by natural selection, *Statistics and computing* 4 (1994) 87–112.
- [2] M. Castelli, L. Trujillo, L. Vanneschi, S. Silva, E. Z-Flores, P. Legrand, Geometric semantic genetic programming with local search, in: *Proceedings of the 2015 annual conference on genetic and evolutionary computation*, 2015, pp. 999–1006.
- [3] L. Muñoz, L. Trujillo, S. Silva, M. Castelli, L. Vanneschi, Evolving multidimensional transformations for symbolic regression with m3gp, *Memetic Computing* 11 (2019) 111–126.
- [4] G. Pietropoli, L. Manzoni, A. Paoletti, M. Castelli, Combining geometric semantic gp with gradient-descent optimization, in: *European Conference on Genetic Programming (Part of EvoStar)*, Springer, 2022, pp. 19–33.
- [5] G. Pietropoli, F. J. Camerota Verdù, L. Manzoni, M. Castelli, Parametrizing gp trees for better symbolic regression performance through gradient descent., in: *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, 2023, pp. 619–622.
- [6] M. Zhang, W. Smart, Genetic programming with gradient descent search for multiclass object classification, in: *European Conference on Genetic Programming*, Springer, 2004, pp. 399–408.
- [7] G. N. Saridis, Toward the realization of intelligent controls, *Proceedings of the IEEE* 67 (1979) 1115–1133.

- 811 [8] P. Antsaklis, Defining intelligent control, report of the task force on
812 intelligent control, IEEE Control Systems Magazine 58 (1994) 4–5.
- 813 [9] F. Marchetti, E. Minisci, A. Riccardi, Towards Intelligent Control via
814 Genetic Programming, Proceedings of the International Joint Confer-
815 ence on Neural Networks (2020). doi:10.1109/IJCNN48605.2020.9207
816 694.
- 817 [10] C.-H. Chiang, A genetic programming based rule generation approach
818 for intelligent control systems, in: 2010 International Symposium on
819 Computer, Communication, Control and Automation (3CA), volume 1,
820 IEEE, 2010, pp. 104–107.
- 821 [11] K. J. Åström, R. M. Murray, Feedback Systems, Princeton University
822 Press, 2010. URL: <http://www.jstor.org/stable/10.2307/j.ctvc4gdk>.
823 doi:10.2307/j.ctvc4gdk.
- 824 [12] C. Utama, B. Karg, C. Meske, S. Lucia, Explainable artificial intel-
825 ligence for deep learning-based model predictive controllers, in: 2022
826 26th International Conference on System Theory, Control and Comput-
827 ing (ICSTCC), 2022, pp. 464–471. doi:10.1109/ICSTCC55426.2022.9
828 931794.
- 829 [13] C. K. Oh, G. J. Barlow, Autonomous controller design for unmanned
830 aerial vehicles using multi-objective genetic programming, in: Proceed-
831 ings of the 2004 Congress on Evolutionary Computation (IEEE Cat.
832 No.04TH8753), 2004, pp. 1538–1545 Vol.2. doi:10.1109/CEC.2004.133
833 1079.
- 834 [14] A. Bourmistrova, S. Khantsis, Genetic Programming in Application
835 to Flight Control System Design Optimisation, New Achievements in
836 Evolutionary Computation (2010).
- 837 [15] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, Neural
838 ordinary differential equations, in: Proceedings of the 32nd International
839 Conference on Neural Information Processing Systems, NIPS’18, Curran
840 Associates Inc., Red Hook, NY, USA, 2018, p. 6572–6583.
- 841 [16] J. L. Lions, Optimal control of systems governed by partial differential
842 equations, volume 170, Springer, 1971.

- [17] F. Neri, C. Cotta, Memetic algorithms and memetic computing optimization: A literature review, *Swarm and Evolutionary Computation* 2 (2012) 1–14.
- [18] X. Chen, Y.-S. Ong, M.-H. Lim, K. C. Tan, A multi-facet survey on memetic computation, *IEEE Transactions on evolutionary computation* 15 (2011) 591–607.
- [19] L. Trujillo, O. Schütze, P. Legrand, et al., Evaluating the effects of local search in genetic programming, in: *EVOLVE-A Bridge between Probability, Set Oriented Numerics, and Evolutionary Computation V*, Springer, 2014, pp. 213–228.
- [20] G. Pietropolli, L. Manzoni, A. Paoletti, M. Castelli, On the hybridization of geometric semantic gp with gradient-based optimizers, *Genetic Programming and Evolvable Machines* 24 (2023) 1–20.
- [21] B. E. Eskridge, D. F. Hougen, Imitating success: A memetic crossover operator for genetic programming, in: *Proceedings of the 2004 congress on evolutionary computation (IEEE Cat. No. 04TH8753)*, volume 1, IEEE, 2004, pp. 809–815.
- [22] P. Wang, K. Tang, E. P. Tsang, X. Yao, A memetic genetic programming with decision tree-based local search for classification problems, in: *2011 IEEE Congress of Evolutionary Computation (CEC)*, IEEE, 2011, pp. 917–924.
- [23] A. Topchy, W. F. Punch, et al., Faster genetic programming based on local gradient search of numeric leaf values, in: *Proceedings of the genetic and evolutionary computation conference (GECCO-2001)*, volume 155162, Morgan Kaufmann San Francisco, CA, 2001.
- [24] G. Nadizar, F. Garrow, B. Sakallioğlu, L. Canonne, S. Silva, L. Vanneschi, An investigation of geometric semantic gp with linear scaling, in: *Proceedings of the Genetic and Evolutionary Computation Conference*, 2023, pp. 1165–1174.
- [25] M. Graff, R. Pena, A. Medina, Wind speed forecasting using genetic programming, in: *2013 IEEE Congress on Evolutionary Computation*, IEEE, 2013, pp. 408–415.

- 875 [26] W. Smart, M. Zhang, Continuously evolving programs in genetic pro-
876 gramming using gradient descent, in: Proceedings of the 7th Asia-Pacific
877 Conference on Complex Systems, 2004.
- 878 [27] M. Kommenda, G. Kronberger, S. Winkler, M. Affenzeller, S. Wagner,
879 Effects of constant optimization by nonlinear least squares minimization
880 in symbolic regression, in: Proceedings of the 15th annual conference
881 companion on Genetic and evolutionary computation, 2013, pp. 1121–
882 1128.
- 883 [28] J. Harrison, M. Virgolin, T. Alderliesten, P. Bosman, Mini-batching,
884 gradient-clipping, first-versus second-order: What works in gradient-
885 based coefficient optimisation for symbolic regression?, in: Proceedings
886 of the Genetic and Evolutionary Computation Conference, 2023, pp.
887 1127–1136.
- 888 [29] F. Marchetti, E. Minisci, Inclusive Genetic Programming, in: T. Hu,
889 N. Lourenço, E. Medvet (Eds.), Lecture Notes in Computer Science
890 (including subseries Lecture Notes in Artificial Intelligence and Lecture
891 Notes in Bioinformatics), volume 12691 LNCS, Springer International
892 Publishing, Cham, 2021, pp. 51–65. doi:10.1007/978-3-030-72812-0
893 _4.
- 894 [30] J. Koza, M. Keane, J. Yu, F. Bennett III, W. Mydlowec, Automatic
895 Creation of Human-Competitive Programs and Controllers by Means of
896 Genetic Programming, Genetic Programming and Evolvable Machines
897 1 (2000) 121–164. doi:10.1023/A:1010076532029.
- 898 [31] C. F. Verdier, M. Mazo, Jr., Formal Controller Synthesis via Genetic
899 Programming, IFAC-PapersOnLine 50 (2017) 7205–7210. URL: [http://www.sciencedirect.com/science/article/pii/S2405896317318](http://www.sciencedirect.com/science/article/pii/S2405896317318979)
900 [979](http://www.sciencedirect.com/science/article/pii/S2405896317318979). doi:<https://doi.org/10.1016/j.ifacol.2017.08.1362>.
- 902 [32] K. Łapa, K. Cpałka, A. Przybył, Genetic programming algorithm for
903 designing of control systems, Information Technology and Control 47
904 (2018) 668–683. doi:10.5755/j01.itc.47.4.20795.
- 905 [33] K. Danai, W. G. La Cava, Controller design by symbolic regression,
906 Mechanical Systems and Signal Processing 151 (2021) 107348. URL:

- 907 <https://doi.org/10.1016/j.ymssp.2020.107348>. doi:10.1016/j.ym
908 ssp.2020.107348.
- 909 [34] G. W. Irwin, K. Warwick, K. J. Hunt, *Neural Network Applications in*
910 *Control*, 1995.
- 911 [35] S. A. Emami, P. Castaldi, A. Banazadeh, Neural network-based
912 flight control systems: Present and future, *Annual Reviews in Con-*
913 *trol* 53 (2022) 97–137. URL: [https://doi.org/10.1016/j.arco](https://doi.org/10.1016/j.arcontrol.2022.04.006)
914 [ntrol.2022.04.006](https://doi.org/10.1016/j.arcontrol.2022.04.006). doi:10.1016/j.arcontrol.2022.04.006.
915 arXiv:2206.05596.
- 916 [36] L. Böttcher, N. Antulov-Fantulin, T. Asikis, AI Pontryagin or how
917 artificial neural networks learn to control dynamical systems, *Nature*
918 *Communications* 13 (2022) 1–9. doi:10.1038/s41467-021-27590-0.
- 919 [37] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization,
920 arXiv preprint arXiv:1412.6980 (2014).
- 921 [38] J. Nocedal, S. J. Wright, *Numerical Optimization*, second ed., Springer,
922 New York, NY, 2006.
- 923 [39] H. J. Kelley, Gradient theory of optimal flight paths, *Ars Journal* 30
924 (1960) 947–954.
- 925 [40] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning* (2016).
- 926 [41] L. S. Pontryagin, E. Mishchenko, V. Boltyanskii, R. Gamkrelidze, *The*
927 *mathematical theory of optimal processes*, 1962.
- 928 [42] Y. Ma, V. Dixit, M. J. Innes, X. Guo, C. Rackauckas, A Compari-
929 son of Automatic Differentiation and Continuous Sensitivity Analysis
930 for Derivatives of Differential Equation Solutions, 2021 IEEE High
931 Performance Extreme Computing Conference, HPEC 2021 (2021) 1–9.
932 doi:10.1109/HPEC49654.2021.9622796. arXiv:1812.01892.
- 933 [43] A. Güneş, G. Baydin, B. A. Pearlmutter, J. M. Siskind, Automatic
934 Differentiation in Machine Learning: a Survey, *Journal of Machine*
935 *Learning Research* 18 (2018) 1–43.

- 936 [44] C. M. Pappalardo, D. Guida, Use of the adjoint method for controlling
937 the mechanical vibrations of nonlinear systems, *Machines* 6 (2018).
938 doi:10.3390/machines6020019.
- 939 [45] S. L. Brunton, J. N. Kutz, Data-Drive Science and Engineering: Machine
940 Learning, Dynamical Systems and Control, volume 53, 2019.
- 941 [46] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy,
942 D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J.
943 van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J.
944 Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng,
945 E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman,
946 I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H.
947 Ribeiro, F. Pedregosa, P. van Mulbregt, SciPy 1.0 Contributors, SciPy
948 1.0: Fundamental Algorithms for Scientific Computing in Python, *Nature Methods* 17 (2020) 261–272. doi:10.1038/s41592-019-0686-2.
949